



TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
PULCHOWK CAMPUS

A  
PROJECT REPORT  
ON  
CHITRAGAMA : CRAFTING THREEFOLD REALMS

**SUBMITTED BY:**

AAGAB BHATTARAI (077BCT001)

AAYUSHA ODARI (077BCT006)

BIPUL NEUPANE (077BCT018)

**SUBMITTED TO:**

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

March, 2024

# Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the project work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head  
Department of Electronics and Computer Engineering  
Pulchowk Campus, Institute of Engineering, TU  
Lalitpur, Nepal.

# Acknowledgments

We would like to express our heartfelt thanks to the Department of Electronics and Computer Engineering at Pulchowk Campus for their invaluable support and encouragement throughout this minor project. We are grateful for the continued support and guidance from our esteemed faculty members and advisors.

We extend our gratitude to Asst. Prof. Santosh Giri and Dr. Sanjeeb Prasad Pandey for their invaluable guidance and insightful feedback throughout the conception and progress of this project. Their support has been instrumental in shaping our project, and we deeply appreciate their continued assistance.

Lastly, we recognize and appreciate the collective efforts, insights, and contributions of everyone involved in this project, directly or indirectly. Your support and collaboration have been pivotal.

Sincerely,

Aagab Bhattarai (077BCT001)

Aayusha Odari (077BCT006)

Bipul Neupane (077BCT018)

# Abstract

Structure from motion(SfM) involves recovering both the three-dimensional structure as well as camera positions(motions) from the given set of images. The project implements SfM pipeline using incremental SfM approach taking as input image set covering multiple angles of a scene and produces as output sparse 3D point cloud and camera positions for the input images. The system starts by pre-processing the given image set to find intrinsic camera calibration parameters by using available image metadata, then proceeds to find feature points in the images. Then, the system finds association between features from different images, these associations are used to select seed image pair from which initial reconstruction is performed. Subsequent images are added by computing camera pose with respect to already reconstructed scene points. Each new addition of image presents opportunity to extend number of scene points if the added image can pair up with images that are already registered with the system and this pair up depends on the fact that the pair should have geometrically verified feature matches between them. The accuracy of the computed scene points is increased by performing Bundle Adjustment; where both scene points and camera pose are jointly refined. When the system has finished incrementally adding the images, it outputs the point cloud along with the camera motions for each of the registered images. Additionally, 3D mesh for the obtained point cloud using Poisson Surface Reconstruction can be obtained. The system was tested to evaluate its performance capabilities; system correctly reconstructs scene points and associated motion in cases where camera motion is well defined(i.e. when there is appropriate relative baseline length between camera projection center) and enough feature correspondence can be computed for the captured scene.

Keywords: *SfM, Incremental SfM, Bundle Adjustment, Point Cloud, Poission Surface Reconstruction*

# Contents

|                                             |          |
|---------------------------------------------|----------|
| Copyright                                   | ii       |
| Acknowledgements                            | iii      |
| Abstract                                    | iv       |
| Contents                                    | vi       |
| List of Figures                             | vii      |
| List of Tables                              | viii     |
| List of Abbreviations                       | ix       |
| <b>1 Introduction</b>                       | <b>1</b> |
| 1.1 Background . . . . .                    | 1        |
| 1.2 Problem statements . . . . .            | 1        |
| 1.3 Objectives . . . . .                    | 1        |
| 1.4 Scope . . . . .                         | 1        |
| <b>2 Literature Review</b>                  | <b>2</b> |
| 2.1 Related Works . . . . .                 | 2        |
| 2.2 Related theory . . . . .                | 2        |
| 2.2.1 Feature Detection . . . . .           | 2        |
| 2.2.2 Two View SfM . . . . .                | 3        |
| 2.2.3 Multiple View SfM . . . . .           | 4        |
| 2.2.4 Surface Reconstruction . . . . .      | 6        |
| <b>3 Methodology</b>                        | <b>8</b> |
| 3.1 Requirement Analysis . . . . .          | 8        |
| 3.1.1 Functional Requirements . . . . .     | 8        |
| 3.1.2 Non-functional Requirements . . . . . | 9        |
| 3.2 Data Collection . . . . .               | 9        |
| 3.2.1 Image Data . . . . .                  | 9        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 3.2.2    | Camera Sensor Data . . . . .                              | 9         |
| 3.3      | Implementation . . . . .                                  | 10        |
| 3.3.1    | Image Input . . . . .                                     | 10        |
| 3.3.2    | Pre-Processing . . . . .                                  | 10        |
| 3.3.3    | Model Initialization . . . . .                            | 11        |
| 3.3.4    | Triangulation and Refinement . . . . .                    | 11        |
| 3.3.5    | New Image Registration . . . . .                          | 12        |
| 3.3.6    | Model Output and Surface Reconstruction . . . . .         | 12        |
| <b>4</b> | <b>Experimental Setup</b>                                 | <b>14</b> |
| 4.1      | Programming Language . . . . .                            | 14        |
| 4.2      | Hardware . . . . .                                        | 14        |
| 4.3      | Processing and Computation . . . . .                      | 14        |
| 4.4      | Visualization and Interfacing . . . . .                   | 15        |
| <b>5</b> | <b>System design</b>                                      | <b>17</b> |
| 5.1      | Use Case Diagram . . . . .                                | 17        |
| 5.2      | Behavioral Diagrams . . . . .                             | 18        |
| 5.3      | Block Diagrams . . . . .                                  | 21        |
| <b>6</b> | <b>Results &amp; Discussion</b>                           | <b>25</b> |
| 6.1      | Testing and Evaluation . . . . .                          | 25        |
| 6.1.1    | Error Threshold Analysis . . . . .                        | 25        |
| 6.1.2    | Average Reprojection Error Analysis . . . . .             | 27        |
| 6.1.3    | Correlation Matrix . . . . .                              | 28        |
| <b>7</b> | <b>Conclusions</b>                                        | <b>30</b> |
| <b>8</b> | <b>Limitations and Future Enhancement</b>                 | <b>31</b> |
|          | References . . . . .                                      | 31        |
| <b>A</b> | <b>More Detail on Feature Extraction</b>                  | <b>34</b> |
| <b>B</b> | <b>Basic Relations For Calculating 3D Points and Pose</b> | <b>36</b> |
| <b>C</b> | <b>Solving Least Square</b>                               | <b>38</b> |
| <b>D</b> | <b>Test Outputs</b>                                       | <b>40</b> |
| <b>E</b> | <b>Using The Software</b>                                 | <b>53</b> |

# List of Figures

|     |                                                                                 |    |
|-----|---------------------------------------------------------------------------------|----|
| 2.1 | Intuitive Illustration of Poisson Reconstruction . . . . .                      | 6  |
| 5.1 | Use Case Diagram . . . . .                                                      | 17 |
| 5.2 | System Sequence Diagram . . . . .                                               | 18 |
| 5.3 | SfM Pipeline Sequence Diagram . . . . .                                         | 19 |
| 5.4 | Activity Diagram . . . . .                                                      | 20 |
| 5.5 | System Overview . . . . .                                                       | 21 |
| 5.6 | Pre-Processing . . . . .                                                        | 22 |
| 5.7 | Model Initialization . . . . .                                                  | 22 |
| 5.8 | Triangulation and Refinement . . . . .                                          | 23 |
| 5.9 | New Image Registration . . . . .                                                | 24 |
| 6.1 | Error threshold vs Average Reprojection Error . . . . .                         | 25 |
| 6.2 | Error Threshold vs BA iterations . . . . .                                      | 26 |
| 6.3 | Error Threshold vs Processing Time . . . . .                                    | 26 |
| 6.4 | Average Reprojection Error across the test run . . . . .                        | 27 |
| 6.5 | Average reprojection error values for erroneous reconstructions . . . . .       | 27 |
| A.1 | Scale-space feature detection using a sub-octave Difference of Gaussian pyramid | 34 |
| A.2 | Computing SIFT Descriptor. . . . .                                              | 35 |
| B.1 | Epipolar geometry for two cameras. . . . .                                      | 36 |
| D.1 | Reconstructed water tower . . . . .                                             | 41 |
| E.1 | Initiating SfM . . . . .                                                        | 53 |
| E.2 | After Completion of SfM . . . . .                                               | 54 |
| E.3 | Mesh Generation File Dialog Boxes . . . . .                                     | 55 |
| E.4 | Mesh Generation Input and Output . . . . .                                      | 55 |

# List of Tables

|     |                                |    |
|-----|--------------------------------|----|
| 6.1 | Correlation Matrix . . . . .   | 28 |
| D.1 | Dataset Output Table . . . . . | 47 |



# List of Abbreviations

|               |                                   |
|---------------|-----------------------------------|
| <b>2D</b>     | 2 Dimension                       |
| <b>3D</b>     | 3 Dimension                       |
| <b>SfM</b>    | Structure from Motion             |
| <b>SIFT</b>   | Scale-Invariant Feature Transform |
| <b>NLoG</b>   | Normalized Laplacian of Gaussian  |
| <b>RANSAC</b> | RANdom SAmple Consensus           |
| <b>BA</b>     | Bundle Adjustment                 |
| <b>PnP</b>    | Perspective-n-Point               |
| <b>EPnP</b>   | Efficient Perspective-n-Point     |
| <b>EXIF</b>   | Exchangable Image File Format     |
| <b>UI</b>     | User Interface                    |
| <b>GUI</b>    | Graphical User Interface          |
| <b>CPU</b>    | Central Processing Unit           |
| <b>GPU</b>    | Graphics Processing Unit          |

# 1. Introduction

## 1.1 Background

In computer vision, creating a 3D representation from 2D photos involves reconstructing the spatial information of objects. Similar to humans leveraging a stereo system with two eyes to perceive depth in the real world, computer vision employs techniques such as stereo vision or structure-from-motion to achieve this. Stereo vision relies on disparities between corresponding points in two images, while structure-from-motion involves reconstructing 3D structures from a sequence of 2D images, estimating camera poses and points in the scene. These methods enable accurate 3D modeling by leveraging the geometric relationships captured in multiple 2D images.

## 1.2 Problem statements

Reconstruction of 3D model from set of 2D images can be used to create experiences such as photo tourism, and also popularly used in visual effects industry. So, we aim to implement algorithmically on a computer what our human perception system does naturally.

## 1.3 Objectives

The primary objective is to build a sparse 3D point cloud of a scene using set 2D images that is input to the system. On completion of primary goals, subsequent focus will be directed towards refining the geometry of the model and improving visual aesthetics.

## 1.4 Scope

The scope of the project is to produce sparse 3D reconstruction for given image set. The project aims to implement simplistic structure from motion pipeline for n-views. To implement n-views SfM, incremental approach to solve the problem is targeted.

## 2. Literature Review

### 2.1 Related Works

In the realm of Structure-from-Motion (SfM) and Multi-View Stereo (MVS), several notable open-source systems contribute to 3D reconstruction. COLMAP[1] provides a comprehensive pipeline with both graphical and command-line interfaces, accommodating ordered and unordered image collections for detailed 3D modeling. Similarly, VisualSFM[2] specializes in large-scale outdoor scene reconstruction, leveraging multi-view stereo techniques for scenarios with extensive visual data. OpenSfM stands out for its simplicity and extensibility, making it popular for educational and prototyping purposes, employing feature matching and bundle adjustment. Lastly, OpenMVG serves as a versatile library with robust feature matching and various pipelines, offering incremental or global methods for sparse reconstruction and bundle adjustment using the Ceres solver. Each of these tools brings distinct features and strengths to the field of 3D scene reconstruction.

### 2.2 Related theory

A short overview of the basic knowledge relating to our proposed project is given below:

#### 2.2.1 Feature Detection

##### Feature Point

Feature point or interest point is any point or region of sufficient variation in intensity or simply a region with dense texture detail. Feature points should have well-defined positions, should be invariant to image rotation and scaling, and also be insensitive to lighting changes.

##### SIFT

SIFT is a robust algorithm used for detecting and describing key features in images. It operates by creating a stack of images, denoted as  $S(x,y,\sigma)$  through convolution with Gaussians of increasing width. In order to apply the NLoG operator to  $S(x,y,\sigma)$ , the difference between all pairs of consecutive images in  $S(x,y,\sigma)$  is found. Then, local extrema in this stack of images is found. To suppress the effects of noise, threshold is used to filter out weak extrema. After obtaining legitimate keypoints, orientation assignment is done, such that for each keypoint it has location, scale and orientation as associated values. SIFT descriptors are computed

for each keypoint. These descriptors are crucial as they uniquely identify keypoints and are highly distinctive and invariant to variations such as changes in viewpoint and illumination. More detail on SIFT can be found in appendix A [3]

## Feature Matching

Feature matching between image pairs can be performed using a brute-force approach, where each feature in one image is compared with every feature in the other image. This involves testing all possible combinations of feature correspondences, making it computationally intensive. The computational complexity of this method is  $O(N * M)$ , where  $N$  and  $M$  are the numbers of features in the respective images. For SIFT descriptors features in one image are matched based on nearest neighbour in another image. Ambiguous matches are filtered out by taking ratio of distance between the closest-match and second-closest match. If the ratio is close to one then such matches are filtered out.

### 2.2.2 Two View SfM

#### Epipolar Geometry

Epipolar geometry relates corresponding points in multiple images, essential for feature matching and camera pose estimation. The epipolar geometry is the intrinsic projective geometry between two views. It is independent of scene structure, and only depends on the cameras' internal parameters and relative pose. Using epipolar geometry we obtain epipolar constraint which results in a single matrix equation relating the internal camera parameters, the relative pose (rotation and translation) of the two cameras and the 3D coordinates of the same point in coordinate systems of two cameras. Further details about epipolar constraint obtained using epipolar geometry is presented in appendix B. [4]

#### 8-Point Algorithm

This algorithm relies on a minimum of eight corresponding points identified in two images for estimating the fundamental matrix. It constructs a linear system of equations utilizing these point correspondences and subsequently solves for the elements of the fundamental matrix. The main advantage of this algorithm is that it is linear, hence fast and easily implemented. However, despite its simplicity, the 8-point algorithm is often criticized for being excessively sensitive to noise. [5]

## **Nister’s Algorithm**

Nister’s 5-point algorithm, proposed by David Nister [6], is a technique that utilizes a minimal set of five corresponding points between two images to compute the essential matrix. This matrix is calculated by generating numerous possible solutions from five-point subsets of matching features. This approach ensures robustness against noise and outliers, through the implementation of random sample consensus (RANSAC).

## **Triangulation**

Given projection matrices, 3D points can be computed from their measured image positions in two or more views utilizing epipolar constraint. This technique to estimate 3D point coordinates from corresponding points in two or more images is triangulation. Ideally, 3D points should lie at the point of intersection of the back-projected rays. However, because of measurement noise, back-projected rays will not generally intersect. Thus 3D points must be chosen in such a way as to minimize an appropriate error metric. Calculating the 3D points given the projection matrices is a fairly trivial task. The mathematical relations used are described in appendix B. [7]

### **2.2.3 Multiple View SfM**

For reconstruction from n-views various approaches like global SfM, and incremental SfM is employed.

#### **Incremental SfM**

Incremental SfM involves a step-by-step process that consists of choosing an initial camera pair for reconstruction and then estimating the 3D structure of the scene. Then successive views are incorporated by localizing new cameras relative to the existing 3D points and estimating new 3D structures. At each step, bundle adjustment is performed to refine camera parameters and reduce reprojection error, ensuring consistency across views. By incorporating bundle adjustment at each step, incremental SfM can produce high-quality reconstructions with accurate camera parameters, leading to more reliable results. However, the multiple instances of bundle adjustment can be computationally expensive and time-consuming, particularly for large datasets. The incremental SfM algorithm may also encounter difficulties when faced with degenerate structures and motion configurations. These challenges arise from geometric configurations within the scene or camera motion that confound standard algorithms, such as scenes with coplanar points or ambiguous depth information resulting from certain camera movements or coincident camera centers. To address these issues, the

incremental SfM approach emphasizes the selection of initial view pairs with substantial baseline lengths between camera centers.

### Perspective-n-Point

Perspective-n-Point is the problem of estimating the pose of a calibrated camera  $P_c$  given a set of  $n$  3D points in the world and their corresponding 2D projections in the image. The camera pose consists of 6 degrees of freedom which are made up of the rotation (roll, pitch, and yaw) and 3D translation of the camera with respect to the world.

A commonly used solution to the problem exists for  $n = 3$  called P3P, and many solutions are available for the general case of  $n \geq 3$  as when  $n = 3$ , the PnP problem is in its minimal form of P3P and can be solved with three point correspondences. However, with just three-point correspondences, P3P yields up to four real, geometrically feasible solutions. For low noise levels, a fourth correspondence can be used to remove ambiguity.[8]

### EPnP

Efficient PnP (EPnP) is a method that solves the general problem of PnP for  $n \geq 4$ . This method is based on the notion that each of the  $n$  points (which are called reference points) can be expressed as a weighted sum of four virtual control points. Thus, the coordinates of these control points become the unknowns of the problem. It is from these control points that the final pose of the camera is solved.

### Bundle Adjustment

Bundle adjustment involves minimizing the reprojection error between the image locations of observed and predicted image points, expressed as the sum of squares of a large number of nonlinear, real-valued functions. This minimization is done using nonlinear least-squares algorithms.

$$E = \sum_j \rho_j(||\pi(P_c, X_k) - x_j||^2)$$

BA jointly performs non-linear refinement of camera parameters  $P_c$  and point parameters  $X_k$  such it that minimizes the reprojection error using a function  $\pi$  that projects scene points to image space which is compared to observed image points  $x_j$  and a loss function  $\rho_j$  to potentially down-weight outliers. [9, 1]

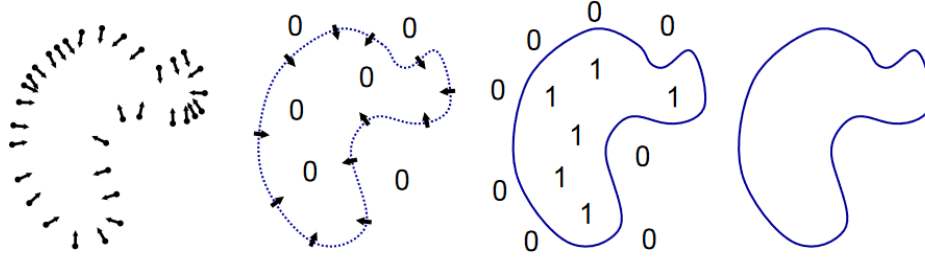


Figure 2.1: Intuitive Illustration of Poisson Reconstruction

## 2.2.4 Surface Reconstruction

### Point Cloud

A point cloud is a discrete set of data points in space. The points may represent a 3D shape or object. Each point position has its set of Cartesian coordinates  $(X, Y, Z)$ . Points can optionally have other parameters, like normals, and such points are called as ‘oriented points’. Point clouds are generally produced by 3D scanners or by photogrammetry software, which measure many points on the external surfaces of objects around them. As the output of 3D scanning processes, point clouds are used for creating 3D models, visualizing, animating, rendering, and mass customization applications. A sparse point cloud is also generally the first output of 3D structure to motion software and can be used as a precursor to full 3D mesh reconstruction.

### Poisson Surface Reconstruction

Although point clouds can be directly interacted with, generally point clouds are converted to polygon mesh or triangle mesh models through the process of surface reconstruction. There are many techniques for converting a point cloud to 3D surface. Some methods build triangles out of existing vertices in point cloud, whereas some methods apply volumetric methods along with marching cubes. Poisson surface reconstruction is one of such methods trying to utilize the benefits of both styles.

Poisson surface reconstruction is quite robust to noise, while still being light on memory requirements and still produces water tight surfaces. The reconstruction algorithm takes in oriented points as input, then considers the underlying surface being described by an implicit function, called as indicator function, which is taken to have the value of zero outside the surface and one inside. As shown in the figure 2.1 taken from the original paper, it then uses the integral relationship between the oriented points and the indicator function to identify surface, as the gradients are zero almost everywhere, except at points near the surface, where they can be represented as the negative of the surface normals sampled in the point cloud.

[10]

## Mesh Filtering

Mesh filtering refers to the process of removing noisy vertices from a mesh. There are various methods of mesh filtering, each producing separate effects. In the simplest method, we simply take as the output vertex, the average of the adjacent vertices in the mesh. This is called the average filter. Mathematically, it can be written as :

$$v_i = \frac{v_i + \sum_{v_n \in \mathbf{V}} v_n}{|\mathbf{V}| + 1}$$

where  $v_i$  is a vertex in the mesh, and  $\mathbf{V}$  is the set of  $n$  closest points of  $v_i$  in the input mesh.



## **3. Methodology**

### **3.1 Requirement Analysis**

#### **3.1.1 Functional Requirements**

##### **Run Structure-from-Motion (SfM) on Images**

The system should take a set of 2D images as input and initiate the structure-from-motion process on the set of images. Upon completion, it should generate a sparse 3D point cloud representing the scene.

##### **Save Camera Path as File**

The system should provide users with the ability to save the camera path, including camera poses and orientations, to a file. In case of any errors during the saving operation, an error message will be displayed to inform the user.

##### **Save Point Cloud on File**

The system should generate point cloud data and allow users to save the generated point cloud representing the 3D scene to a file for future reference.

##### **Reconstruct 3D Surface**

The system can perform surface reconstruction on the generated point cloud to create a 3D surface model of the scene. This process enables users to visualize the scene in a more realistic and detailed manner.

##### **Write 3D Data to File**

Users should be able to write the reconstructed 3D data, including both the point cloud and the surface model, to a file. This functionality facilitates the storage and sharing of the 3D scene information in a standardized format.

## **Read 3D Data from File**

The system should support the loading of previously saved 3D data from a file. Users can import the stored 3D scene information, including point cloud and surface model data, for visualization or further processing within the system.

### **3.1.2 Non-functional Requirements**

#### **Performance**

System should be fairly efficient in terms of time to construct a sparse scene.

#### **Extensibility**

System should be built with aim for extensibility, so modularity should be aimed.

#### **Usability**

The user interface should be intuitive and user-friendly to ensure ease of use for both novice and experienced users

## **3.2 Data Collection**

### **3.2.1 Image Data**

The image sets we used for this project was acquired through the collection developed by Carl Olsson of Lund University, which can be accessed from the following link:

<https://www.maths.lth.se/matematiklth/personal/calle/dataset/dataset.html>.

The dataset consists of over 70 sets of images of scenes each consisting of a count of 10 images to 300. Among the images provided there, 49 of the image sets containing of over 2511 images were successfully processed, preferring the image sets with lower image count. Even then for some scenes, only partial dataset was used due to computational constraints. Thus each image set used in the project consisted of images not exceeding 80. The system characteristics can be properly tested and evaluted from the data obtained from the resource, as the resource consists of image set capturing diverse scene structure along with diverse set of camera motion.

### **3.2.2 Camera Sensor Data**

Camera sensor database was taken from openMVG project repository. This repository contains a list of camera model and their corresponding camera sensor size.

[https://github.com/openMVG/CameraSensorSizeDatabase/blob/master/sensor\\_datab](https://github.com/openMVG/CameraSensorSizeDatabase/blob/master/sensor_datab)

## 3.3 Implementation

### 3.3.1 Image Input

Set of images of a scene or an object is fed into the system as input. Here, images are assumed to be taken from a single camera, so the system only deals with only one set of camera intrinsic parameters. In addition to that, the distortion coefficient of the camera is assumed to be zero.

### 3.3.2 Pre-Processing

#### 1. Calculation of intrinsic camera parameters

The EXIF tag from the image is used to calculate calibration matrix  $K$  for the camera. The metadata typically contains the focal length  $f_m$  of the camera lens, given in millimeters. This is used to obtain value of the focal length in pixels  $f_p$ :

$$f_p = \frac{f_m w_i}{w_s}$$

where,  $w_i$  and  $w_s$  are the size of the image in pixels and camera sensor(in millimeters) respectively. The sensor size for a particular camera is obtained from the camera sensor database indexed by the camera model.

Assuming zero skew, the calibration matrix  $K$  is:

$$K = \begin{bmatrix} f_x & 0 & \frac{w_i}{2} \\ 0 & f_y & \frac{h_i}{2} \\ 0 & 0 & 1 \end{bmatrix}$$

#### 2. Computation of SIFT features

For each image path provided to the system, the corresponding image file is loaded and SIFT features, i.e. location of each key point and corresponding descriptor are computed and stored.

#### 3. Scene Graph Initialization

For all possible image pair combinations following steps are performed:

- **Feature Matching:** For each feature in the first image (query image) two feature matches in the second image (train image) is found. Then, Lowe's ratio test is performed, wherein matches whose first closest match that is within  $x\%$  distance of the second closest match is considered as a good match, these matches are still considered as putative/tentative correspondences.

- **Relative Pose Recovery:** For image pair that have minimum threshold of putative correspondences, RANSAC method is employed to compute essential matrix and at the same the matches are distinguished as inliers and outliers based on their distance to the epipolar line. Then, essential matrix is decomposed to get relative pose between two views. If the number of inlier correspondences is below a certain threshold, then it is assumed that the correspondence set for the view pair is geometrically degenerate, so the view pair association is marked such that it won't be used in the later parts of the pipeline.
- **Median triangulation Angle Computaiton:** For each view pair with verified matches, a subset of matches are sampled and triangulated. For each triangulated point, triangulation angle, i.e. angle between rays originating from each camera's position to the triangulated point is computed. Then, median of the triangulation angles is computed and stored.
- **Addition to Scene Graph:** Data associated with each view pair, is organized and stored. For the features that are found to be unique, a global descriptor value is associated with it.

### 3.3.3 Model Initialization

#### 1. Inital image pair selection

All image pairs with the median triangulation angle above the defined acceptable median triangulation angle are considered and the image pair with the highest number of feature correspondence among the features that can be seen by maximum number of images is selected for initialization.

#### 2. Extrinsic Pose Setup

For the selected initialization view ID, one of the camera positions is considered to be at the center of the world coordinate system. The extrinsic parameter of the other view is then set equal to the relative pose transformation between these two views.

### 3.3.4 Triangulation and Refinement

#### 1. Object Point Computation

For a given pair of views, object point is triangulated. The color of the triangulated point is assigned based on the color of pixel at the feature point location. The triangulated points are filtered for potential outliers using statistical outlier filtering technique. Here, mean distance of each point to it's closest n-neighbours is computed and if the mean distance to closest n-neighbours isn't within a standard deviation of mean of

mean distances for all the points, then the point is considered as outlier and is filtered out.

## 2. Reprojection Error Calculation

From the object points that was triangulated, the object points are projected onto each image plane using external pose and internal camera calibration matrix. The difference between reprojected object point and corresponding observed image point is calculated for all the feature points. Based on this error reconstruction performance is evaluated and if the reprojection error is considerably high bundle adjustment is performed.

## 3. Bundle Adjustment

Object points and the camera parameters are jointly refined to minimize the reprojection error. Here, all object points and camera parameters computed by the system until the refinement call is considered for the refinement process.

### 3.3.5 New Image Registration

#### 1. New View Selection

Image view that sees the maximum number of object points that has been triangulated is selected as the next to be registered by the system.

#### 2. 2D to 3D Correspondence

For the new view that is to be registered, the keypoint location associated to the new view and object point correspondence is established by leveraging global descriptor assigned for each keypoint.

#### 3. Extrinsic Pose Recovery

Using 2D to 3D correspondence, external camera parameters for the newly registered image with respect to the initially defined coordinate system are computed using the PnP algorithm. After computing the extrinsic camera parameters the image view is said to be registered.

#### 4. Pair up with registered image

For all possible combination of image pairs with acceptable median triangulation angle and valid number of correspondences that haven't been triangulated are sent as input to Triangulation and Refinement part of the system.

For all of the remaining views, the process of "New Image Registration" and "Triangulation and Refinement" repeats.

### 3.3.6 Model Output and Surface Reconstruction

The sparse 3D point cloud as well as the camera path for all input images are obtained in the form of a .ply file. It consists of both 3D vertices and estimated normals for those vertices,

along with color sampled from some of the original image. This file can be opened using any 3D viewing software and library.

The file can also be opened again to attempt surface reconstruction on the point cloud output of the system. Poisson surface reconstruction is used as the method for reconstructing the surface. There are three parameters used to control the surface reconstruction result. The first one is the ‘depth’ parameter, which controls the depth of the octree used internally by the Poisson reconstruction to store the point cloud vertices in. The second parameter is a density filter parameter, which is the threshold used to discard the extraneous surface built around the low density vertices. The final parameter is used to control the number of iterations we go through when we apply filtration on the generated mesh, using simple average filter. The generated mesh is also written again in a .ply file which can be viewed in external programs.

## 4. Experimental Setup

### 4.1 Programming Language

Python was chosen to be our language for implementation. The version of python that we used for conducting this project was 3.11. Python is a high-level, interpreted programming language known for its clear syntax and readability, which made it an excellent choice for such a project. Also the extensive availability of image processing and numerical libraries in python aided us vastly in timely completion of our project.

### 4.2 Hardware

Computer vision, at its core is a computational heavy job which requires quite some time to be invested towards processing of the images. For this project, we utilized only CPU based processing, as it was simpler to setup compared to GPU based processing, as well as many problems were better suited to CPU based sequential processing. We accomplished this using various libraries, and as such the choice was mostly out of our hands, though the libraries did use multi-threaded algorithms at various points, thus utilizing available CPU cores to the full. We ran the processing both on our laptops as well as in the virtual machines provided for free online. We used our laptops and google collab for generating the 3D object files to view, whereas we used Azure virtual machines to run and test our system against a wide ranging Lund University dataset as a part of testing and evaluation process.

### 4.3 Processing and Computation

We used the following libraries, all in python for fulfilling our image processing as well as numerical computational needs:

1. **Numpy:**

We used NumPy for its powerful matrix and array objects manipulations and comprehensive mathematical functions. Numpy's compatibility with other Python libraries streamlined our workflow, proving essential for the project's completion. The efficient handling of large data arrays was particularly beneficial for processing the large amount of data in this interpreted language.

2. **SciPy:**

SciPy library is a fundamental package for scientific computing in Python. SciPy

provides numerous efficient and user-friendly interfaces for tasks such as numerical integration, interpolation, optimization, linear algebra, and more. Specifically for 3D reconstruction, we used modules as the ndimage module, which contains various functions for multi-dimensional image processing, as well as the linalg module for solving linear systems of equations.

### 3. **OpenCV:**

OpenCV, an open-source library used for real-time image processing, was another key tool in this project. Its vast array of features and algorithms, particularly those related to image analysis, feature extraction and correspondence matching, made it ideal for processing and manipulating our data. We also relied upon OpenCV for providing us with various implementations of the algorithms we had to use, without which we would have certainly resulted in a software with much poorer performance.

### 4. **Open3D:**

Although not used extensively, Open3D library certainly aided us in the post SfM steps. Open3D is a 3D point cloud and mesh processing library which we used to process the output point cloud of our SfM pipeline. By creating triangle meshes, Open3D provided a more comprehensive representation of our reconstructed scenes, allowing us to better validate our SfM pipeline through visual inspection.

## 4.4 Visualization and Interfacing

We used an external software and multiple python libraries for visualization the results and interfacing our software with user:

### 1. **MeshLab:**

MeshLab is an open source system for processing and editing 3D triangular meshes. It actually itself provides a multitude of tools for editing, cleaning, healing, inspecting, rendering, texturing and converting meshes, but we only utilized as an external program to view the 3D output files generated by our software.

### 2. **Matplotlib:**

Matplotlib is a popular graph visualization tool used extensively for scientific purposes. We utilized specifically its 2D graph plotting modules for visualizing various statistical parameters in various phases of our development. In the initial phases of development, we also utilized its 3D plotting module to visualize the formation of point cloud data, although later we moved on to various other tools.



### 3. **PyVista:**

PyVista is a python library targeted at providing an easy method of visualizing and interacting with 3D data, both point cloud and meshes. Actually Open3D also provided us with a basic 3D visualization interface, but we found pyvista to be more easy to use and integrate with our existing workflow. PyVista is based directly upon another visualization library named VTK (The Visualization Toolkit).

### 4. **Tkinter:**

Tkinter is the GUI library provided by default in python implementations. We used this library solely because of our very light UI requirements and its availability in all current python implementations.

# 5. System design

## 5.1 Use Case Diagram

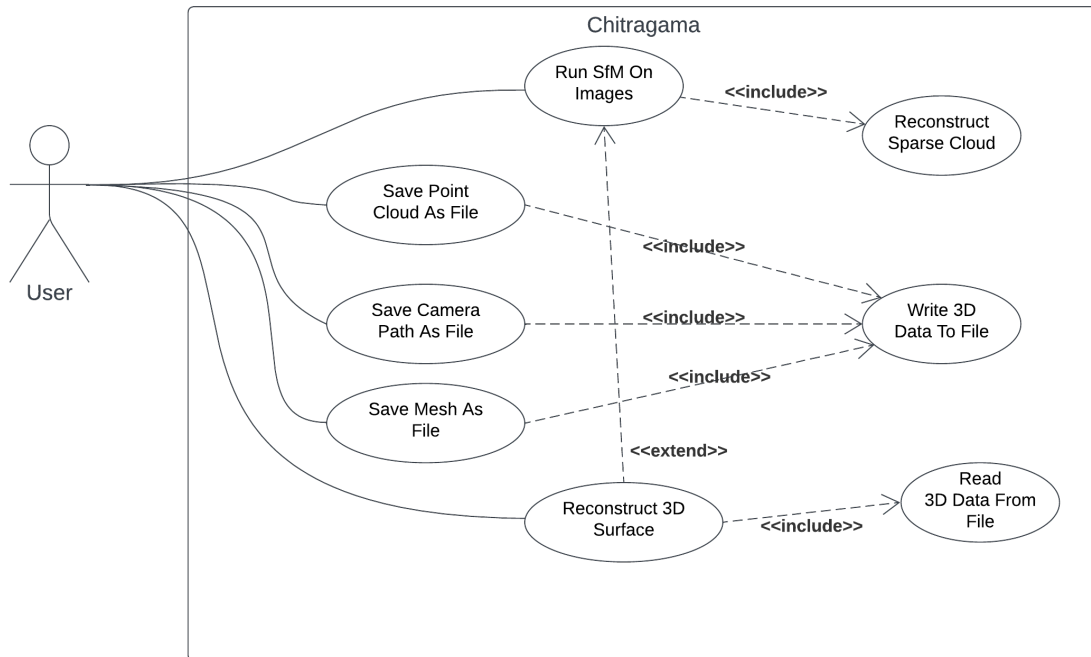


Figure 5.1: Use Case Diagram

## 5.2 Behavioral Diagrams

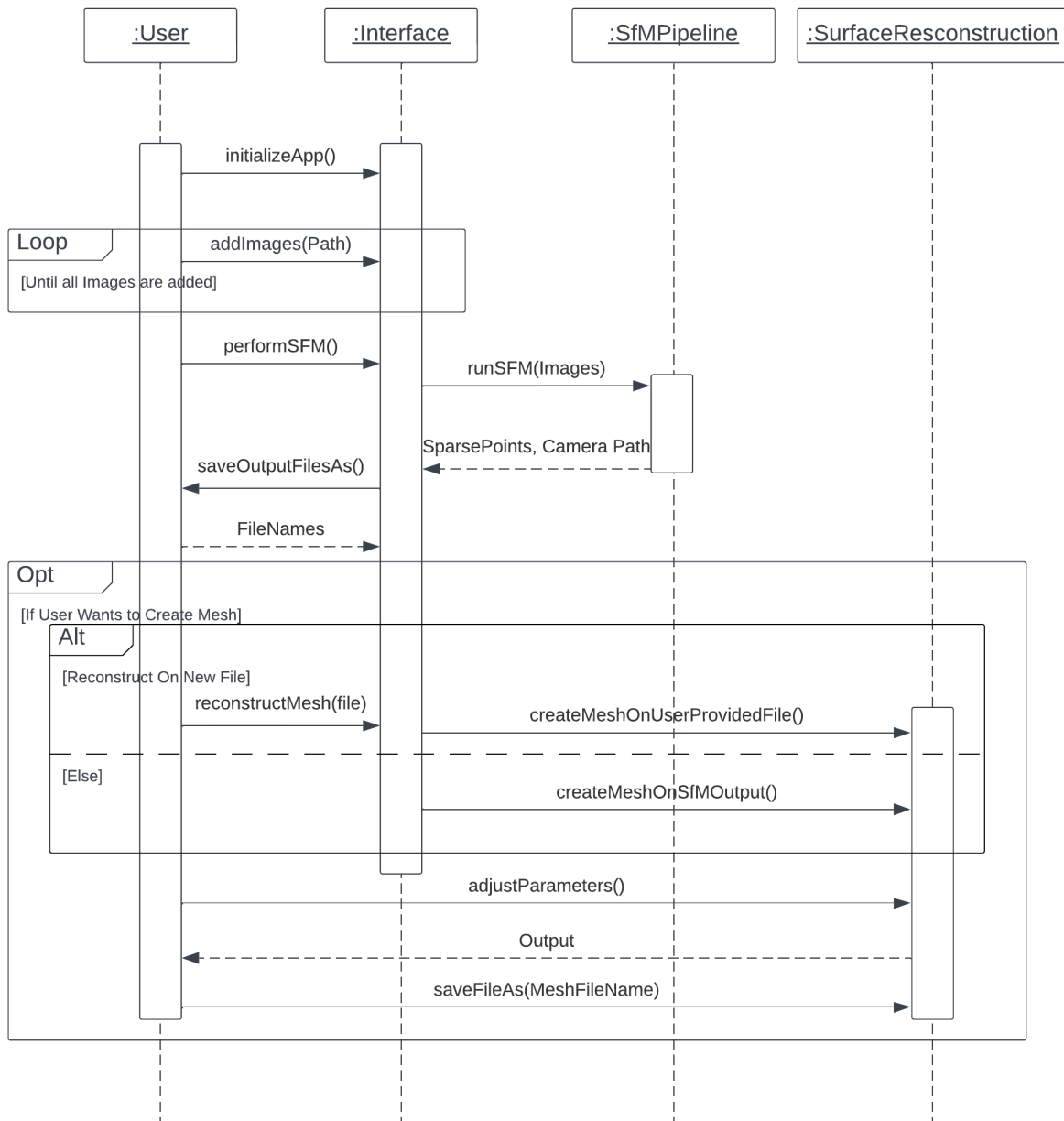


Figure 5.2: System Sequence Diagram

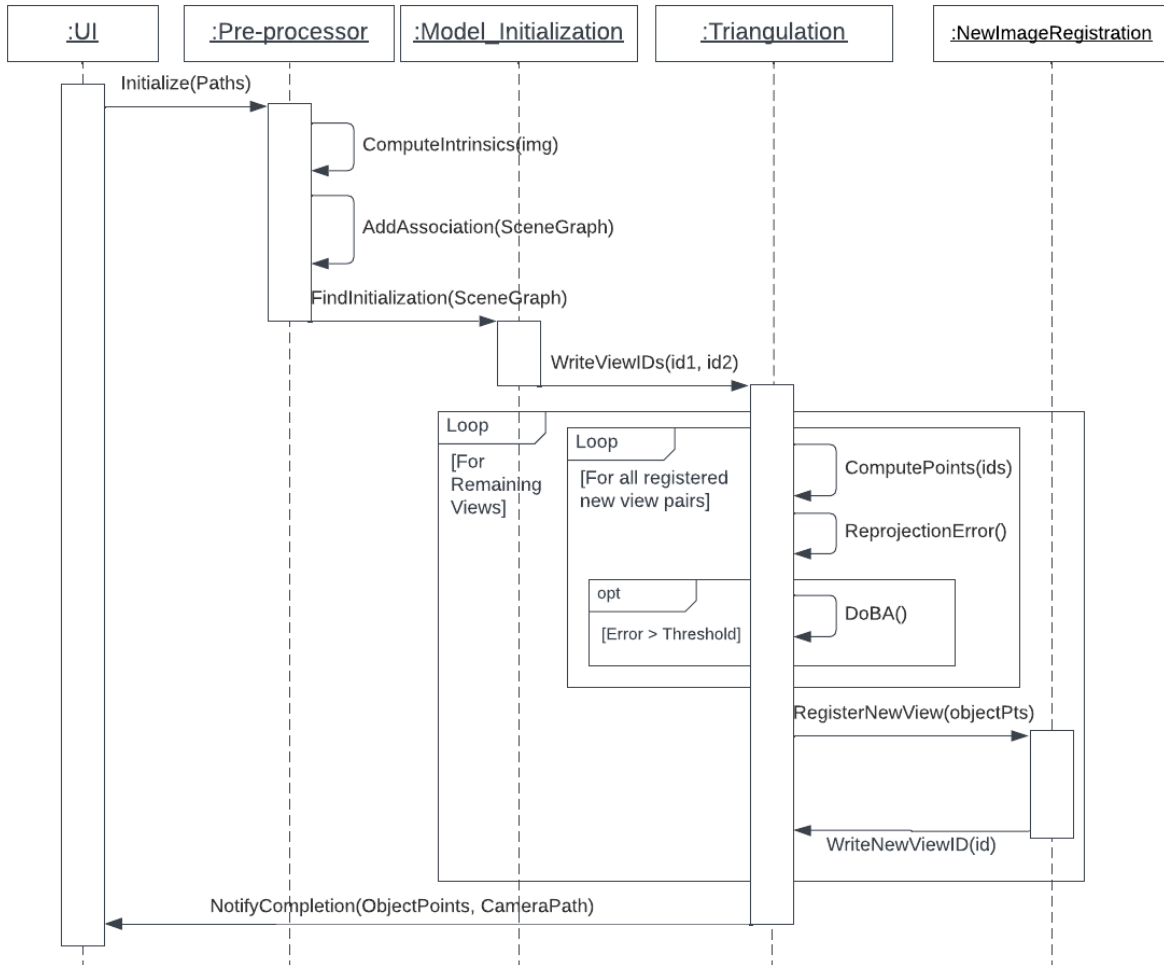


Figure 5.3: SfM Pipeline Sequence Diagram

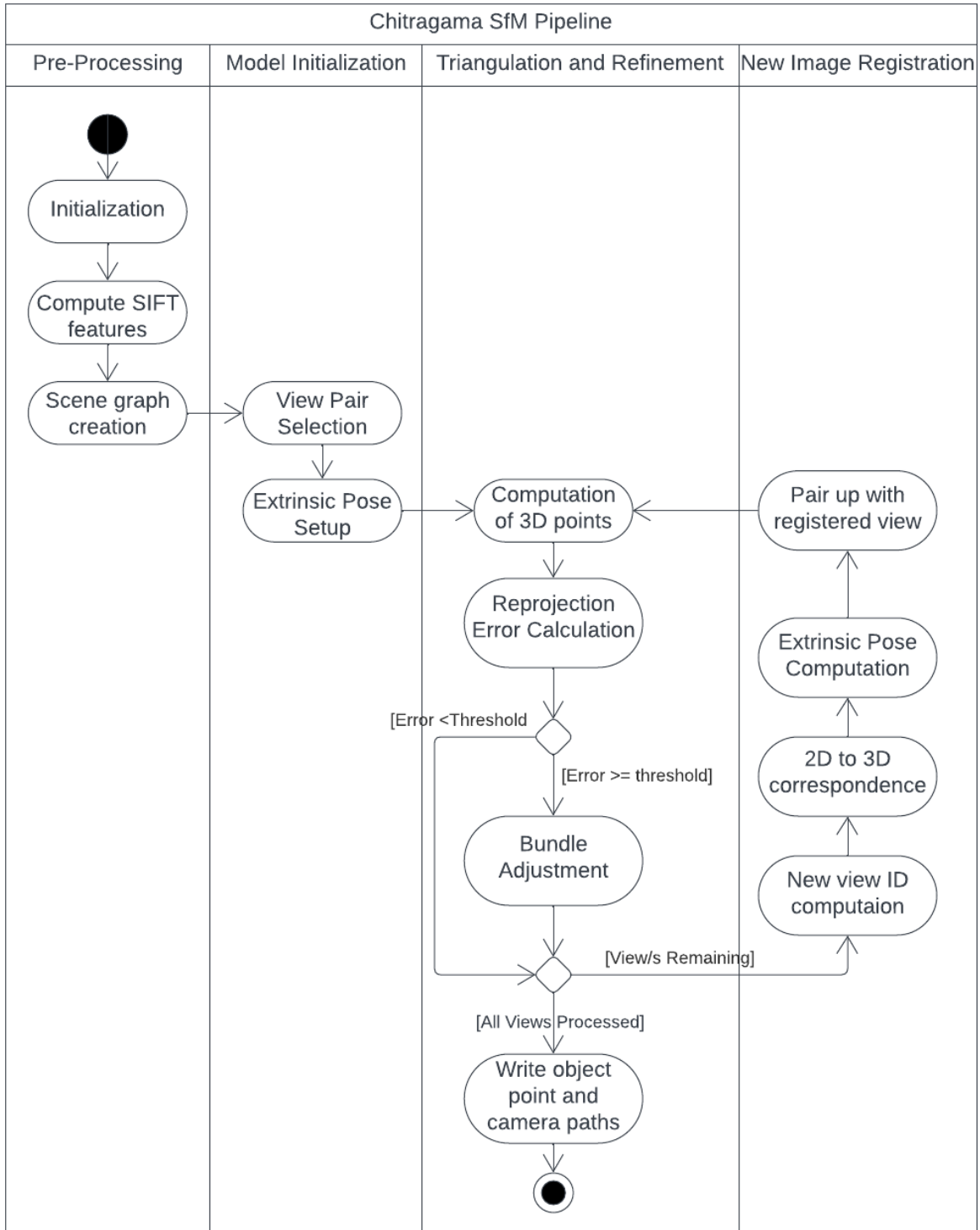


Figure 5.4: Activity Diagram

## 5.3 Block Diagrams

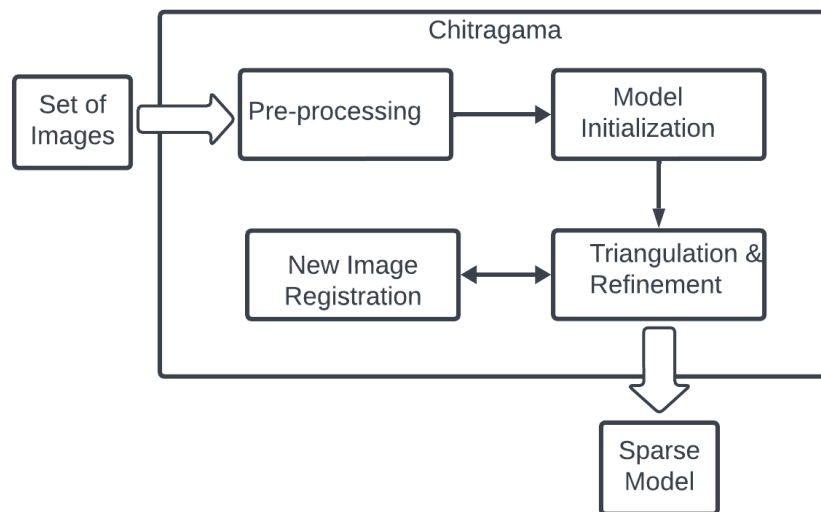


Figure 5.5: System Overview

In Figure 5.5, the system overview for the SfM pipeline is presented, illustrating the overall architecture and components involved. For the image set given to the system, it is pre-processed to establish relation between different image pairs. Model initialization selects the seed image pair for reconstruction. The remaining image views are incrementally registered and provided for reconstruction. When all images are registered the system outputs a sparse object model along with the camera positions for the registered images.

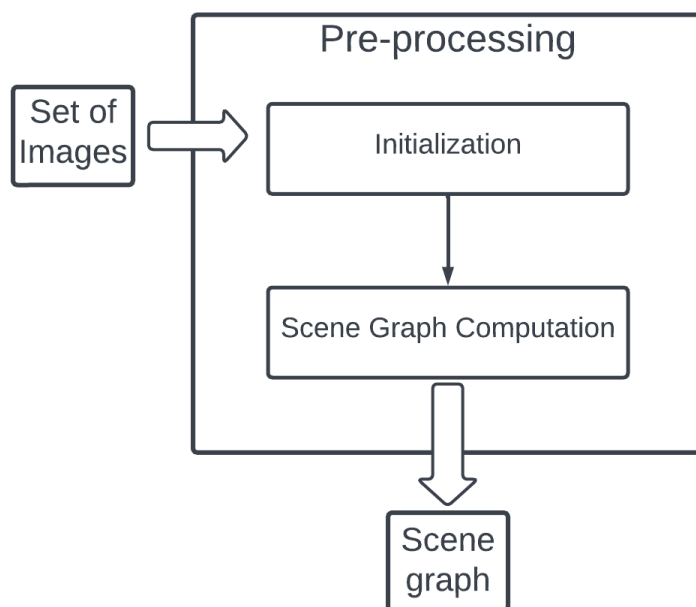


Figure 5.6: Pre-Processing

As shown in the figure 5.6, pre-processing involves initialization and scene graph computation to give as output the scene graph. Initialization step constitutes computation of intrinsic camera parameters and feature extraction. Scene graph computation step constitutes computing and storing the nature of all possible image pairs.

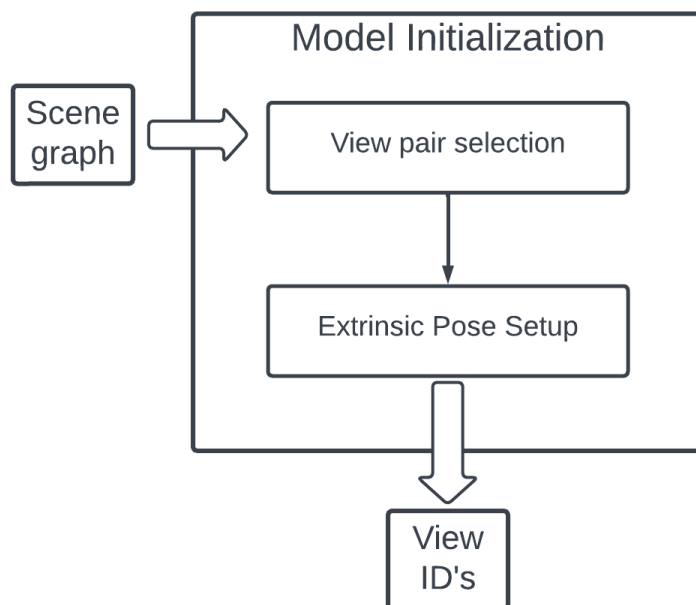


Figure 5.7: Model Initialization

As shown in the figure 5.7, this part of the system takes as input the scene graph and

from it, finds image pair appropriate for initial reconstruction. For the seed image pair it establishes extrinsic camera pose for the view pair and outputs the view id's corresponding to the seed pair.

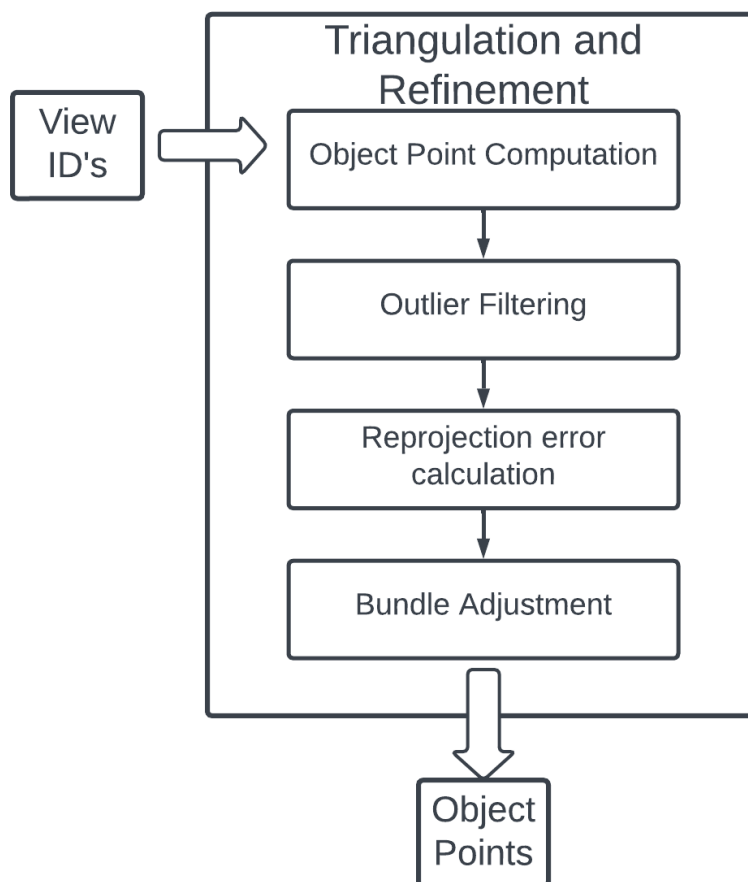


Figure 5.8: Triangulation and Refinement

As shown in the figure 5.7, for the view ID's provided as input, feature matches for the corresponding views that aren't already in the object points model are reconstructed. Local outlier filtering is performed to identify and remove any outlier in the triangulated points. Reprojection error for newly triangulated points is computed and if accumulated reprojection error exceeds a certain threshold object points and the extrinsic camera parameters are refined.



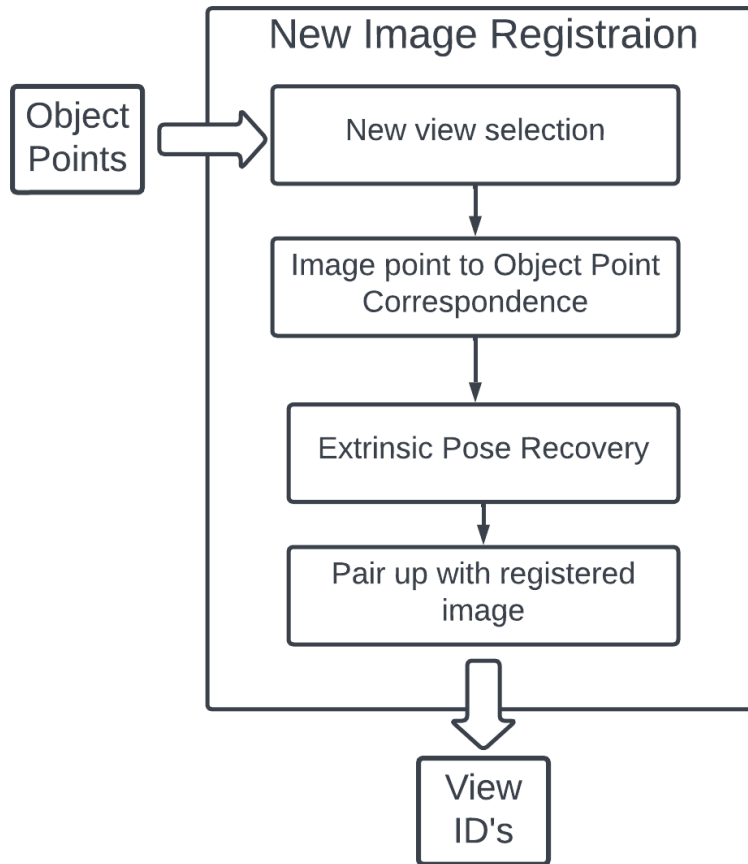


Figure 5.9: New Image Registration

As shown in the figure 5.7, for views aren't processed, view that sees most number of triangulated objected point is chosen. Correct correspondence between image points and object points are computed and extrinsic camera parameters is determined. The new view is paired up with images that have already been registered to potentially triangulate new object points.

## 6. Results & Discussion

### 6.1 Testing and Evaluation

The system implementation was tested using image sets that was discussed in the section of Data collection. Here 49 number of image sets were used, containing a total of 2511 separate images. Each image set was run multiple times while varying error threshold parameter along with resolution of images. Following are the results for the test run as seen in the table in appendix D.

#### 6.1.1 Error Threshold Analysis

As the system performs bundle adjustment when accumulated error crosses the error threshold, so choice of error threshold affects average reprojection error and processing time which is shown by the following graphs:

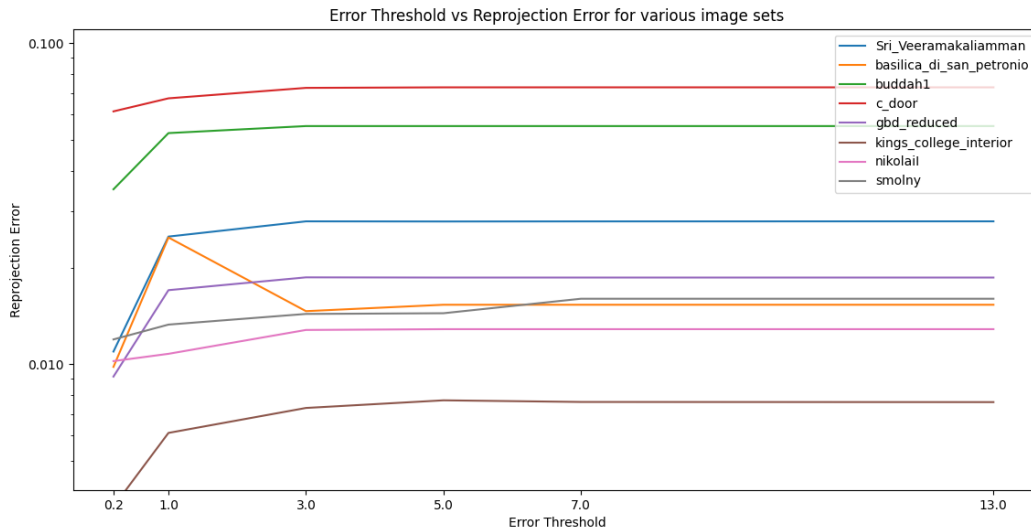


Figure 6.1: Error threshold vs Average Reprojection Error

Figure 6.1 illustrates the relationship between error threshold and average reprojection error. As shown in the graph as tolerance for error, i.e. error threshold increases, the average reprojection error also increases.

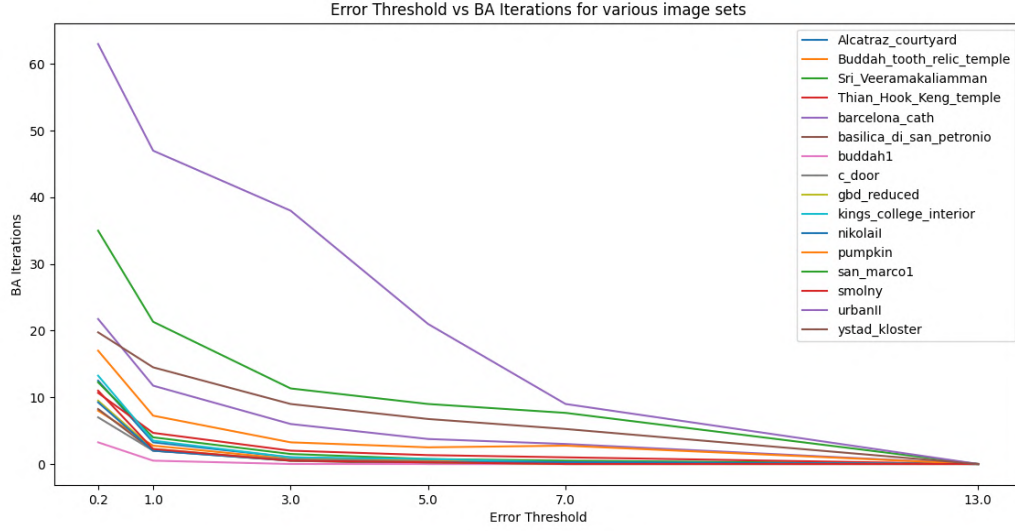


Figure 6.2: Error Threshold vs BA iterations

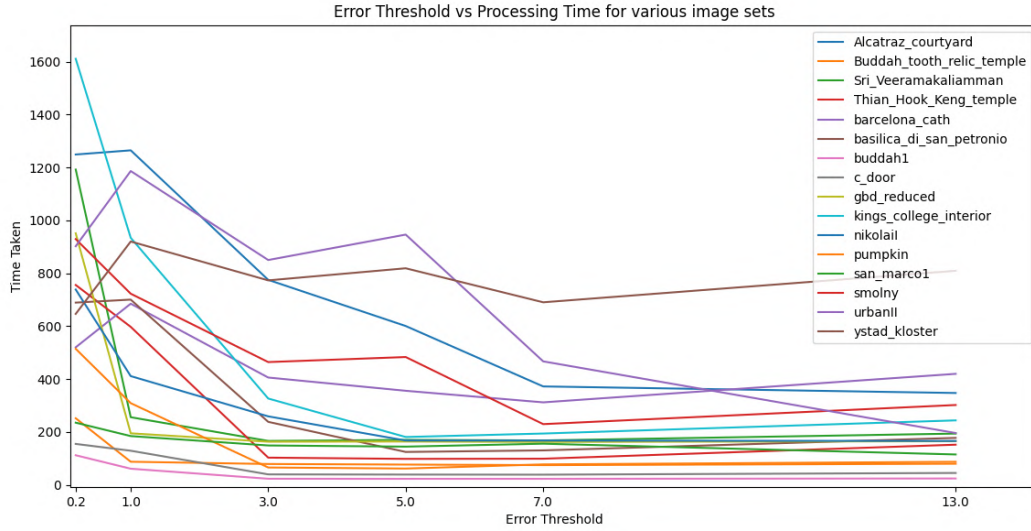


Figure 6.3: Error Threshold vs Processing Time

As shown in figure 6.3 for low values of error threshold time required for reconstruction is very high compared to higher threshold values, this fact can also be inferred from the figure 6.2, which shows that for lower error threshold values system eagerly performs bundle adjustment, while for higher values of error threshold no bundle adjustment is performed. So, there is a trade-off between processing time and average re-projection error for values of error threshold.

## 6.1.2 Average Reprojection Error Analysis

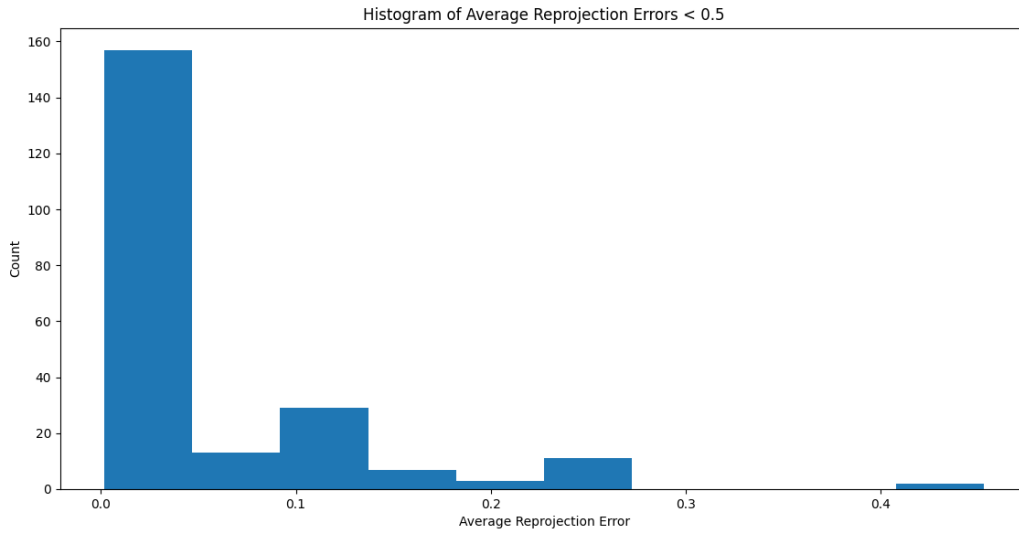


Figure 6.4: Average Reprojection Error across the test run

Figure 6.4 shows histogram plot for average reprojection error, where count denotes number of runs of reconstruction. In most of the runs reconstruction was successfully performed with low value of average reprojection error. Some examples of the reconstruction is shown in appendix D.

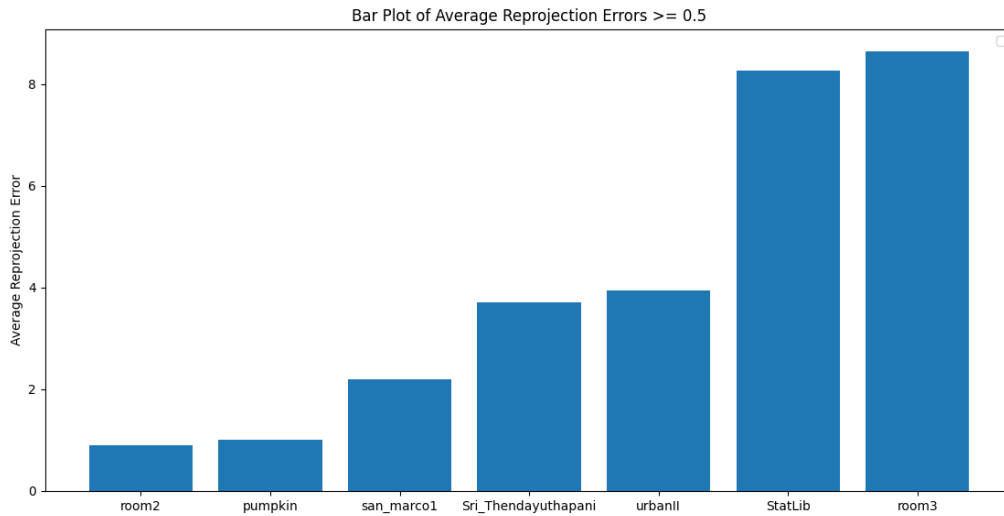


Figure 6.5: Average reprojection error values for erroneous reconstructions

Figure 6.5 shows the average reprojection error for various image sets in which the system

failed to perform proper reconstruction. Upon analyzing the input images for these erroneous reconstructions, a common theme emerges: these images feature clustered camera poses, indicating a narrow distance between camera centers. Another significant error source is the presence of low parallax in the captured scene structure, indicating that the images capture structures located far from the camera. The system also struggled when the captured structure consisted of repeating patterns within the structure, making it difficult for the system to properly associate features across multiple images. Additionally, visual artifacts due to capturing moving objects were observed, which likely contributed to the increase in error values. The erroneous reconstruction results are shown in the appendix D.

### 6.1.3 Correlation Matrix

|                            | Image<br>Count | Pixels | Processing<br>Time | 3D<br>Points | Error<br>Thresh-<br>old | Average<br>Repro-<br>jection<br>Error |
|----------------------------|----------------|--------|--------------------|--------------|-------------------------|---------------------------------------|
| Image Count                | 1.0000         | -      | 0.1346             | 0.1470       | -                       | 0.0674                                |
| Pixels                     | -              | 1.0000 | 0.4028             | 0.7246       | -                       | 0.0673                                |
| Processing Time            | 0.1346         | 0.4028 | 1.0000             | 0.5779       | -0.1997                 | 0.0128                                |
| 3D Points                  | 0.1470         | 0.7246 | 0.5779             | 1.0000       | -0.0099                 | 0.0575                                |
| Error Threshold            | -              | -      | -0.1997            | -0.0099      | 1.0000                  | 0.0160                                |
| Average Reprojection Error | 0.0674         | 0.0673 | 0.0128             | 0.0575       | 0.0160                  | 1.0000                                |

Table 6.1: Correlation Matrix

Table 6.1 summarizes relation between various parameters. Increasing number of images means increasing processing time as well as increases number of 3D points that can be triangulated and also introduces more sources of error, so positive correlation is shown between image count and total time, points , reprojection error.

Similarly resolution increase, i.e. increase in total pixel count increases processing time, and at higher resolution more features can be extracted and as a result more 3D points can be triangulated and added to the reconstructed scene. Similar to increasing image count, increase in number of pixel leads to increase in reprojection error because of added data.

The positive correlation between total time and reprojection error is explained by the fact that system has to perform higher number of bundle adjustments when reconstruction has some error associated with it. Similarly, negative correlation with error threshold has already been explained in previous section of Error Threshold Analysis.

Postive correlation between error threshold and average reprojection error(also explained

earlier), simply highlights the fact that increasing error threshold causes number of bundle adjustments performed to drop which means reconstructed points are not refined which leads to higher average reprojection error. Very weak negative correlation between error threshold and 3D points is explained by the fact that on smaller error threshold values 3D points are refined so as a result some points after refinement are spared by outlier filtering.

## 7. Conclusions

The Chitragama project finally concludes as a success in implementing a Structure from Motion (SfM) pipeline using the Incremental SfM method. In scenarios with an adequate number of images featuring varying translations, our pipeline provides reasonably accurate camera pose estimation and sparse point cloud generation. We acknowledge that our initial goal of generating a sparse point cloud was achieved, although the inherent difficulty of estimating motion and structure from noisy images becomes apparent in smaller datasets or those with limited variation.

In our pursuit of a simplistic yet functional SfM pipeline, we concede that our software may lack certain features compared to more comprehensive tools in the market. Nevertheless, we have successfully met the outlined functional requirements, producing a sparse 3D point cloud stored in a file with the appropriate extension (.ply). An optional feature has been incorporated to generate basic 3D meshes from the SfM pipeline's output, aiding in visual validation.

This project has been a valuable learning experience in fundamental concepts of image processing and computer vision. Although our exploration might not match the depth of some endeavors, we appreciate the opportunity to delve into various technologies related to numerical processing. Our teamwork and project management skills have been honed, and we recognize that continuous learning and improvement are integral aspects of our journey.

## 8. Limitations and Future Enhancement

The limitations for the current system implementation are:

- **Time Complexity:** Present system requires considerable amount of time to complete the reconstruction process when performing bundle adjustment. Immediate future enhancement would be to optimize implementation code to reduce time taken to perform reconstruction.
- **Input image Number consideration:** System cannot efficiently manage memory and handle computation appropriately when number of input images is large.
- **Camera Calibration Parameters:** The SfM pipeline requires intrinsic cameras calibration parameters to perform reconstruction. This requirement is automatically satisfied for images with sufficient information in the EXIF tag and whose camera sensor size information is present in camera sensor size database. So, major limitation is in the fact that known intrinsic camera calibration parameters is a strict requirement.

Possible future enhancement for the system are:

- **Dense Reconstruction:** The obvious next step to follow is dense reconstruction. We saw in some of our outputs of surface reconstruction which were not always convincing. Generally this surface reconstruction is accompanied with dense reconstruction where the results of our SfM are reused and a depth map of the scene is first developed, which later aids in the development of detailed reconstruction.
- **Packaged Product:** Currently we are using multiple libraries to develop, run and visualize the point cloud from our SfM pipeline. We could further improve upon this by creating an independent visualization component.
- **Integrating Multiple Cameras:** Our current implementation runs assuming the images were taken using the same camera. As an enhancement, we could certainly attempt to integrate multiple images taken from multiple camera, which would provide us with a much wider data to take the samples for reconstruction.



# References

- [1] Johannes L. Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [2] Changchang Wu. Towards linear-time incremental structure from motion. In *2013 International Conference on 3D Vision - 3DV 2013*, pages 127–134, 2013.
- [3] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [4] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, New York, NY, USA, 2 edition, 2003.
- [5] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2010.
- [6] D. Nister. An efficient solution to the five-point relative pose problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(6):756–770, 2004.
- [7] Richard Hartley and Peter Sturm. Triangulation. *International Journal of Computer Vision*, 25(2):107–123, 1997.
- [8] Martin A. Fischler and Robert C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun. ACM*, 1981.
- [9] Bill Triggs, Peter F McLauchlan, Richard I Hartley, and Andrew Fitzgibbon. Bundle adjustment: A modern synthesis. *Vision algorithms: Theory and practice*, 2000:298–372, 2000.
- [10] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Eurographics Symposium on Geometry Processing*. The Eurographics Association, 2006.
- [11] Olivier Faugeras. Three-dimensional computer vision: a geometric viewpoint. *MIT press*, 01 1993.

- [12] Peter J. Huber. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1):73 – 101, 1964.

## Appendix A

### More Detail on Feature Extraction

#### SIFT Detection

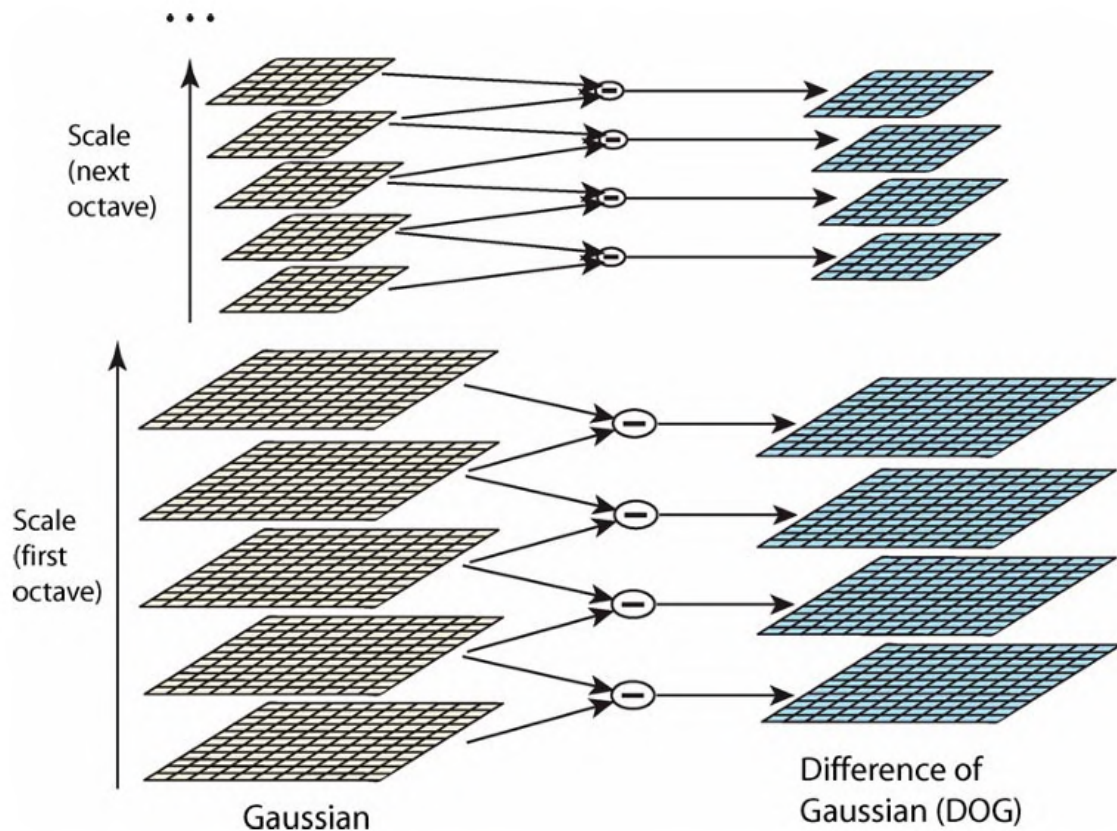


Figure A.1: Scale-space feature detection using a sub-octave Difference of Gaussian pyramid

Image Source: David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004. [3]

For each octave of scale space, the initial image is repeatedly convolved with Gaussians to produce the set of scale space images shown on the left. Adjacent Gaussian images are subtracted to produce the difference-of-Gaussian images on the right. After each octave, the Gaussian image is down-sampled by a factor of 2, and the process repeated.

## SIFT Description

In the SIFT algorithm, the gradient orientation is instrumental in creating distinctive descriptors for keypoints. When a keypoint is detected, the gradient is computed at each pixel within a  $16 \times 16$  window. To capture the local structure, a weighted gradient orientation histogram is formed in each  $4 \times 4$  quadrant. This histogram represents the distribution of gradient orientations, with eight bins capturing different directions. The gradients are weighted using a Gaussian fall-off function to reduce the impact of distant gradients. The resulting 128-dimensional vector, formed by concatenating the histogram values from all quadrants, serves as a unique and invariant descriptor for the keypoint. Normalization and clipping steps enhance robustness to variations, yielding a final descriptor that is distinctive and invariant under various conditions.

The characteristic scale of the feature is used to rescale the image patch that represent the feature. Also, the magnitude of the gradient is ignored and as it is subject to lighting conditions as well, so we end up being invariant lighting conditions. To address challenges related to orientation variance, the principal orientation of the feature is computed. The gradient direction corresponding to the maximum value in the histogram is what we refer to as the principal orientation of the feature. So, we use principal orientation to reorient the feature to point to a specified direction.

Then, after this we can compare descriptors to match various features.

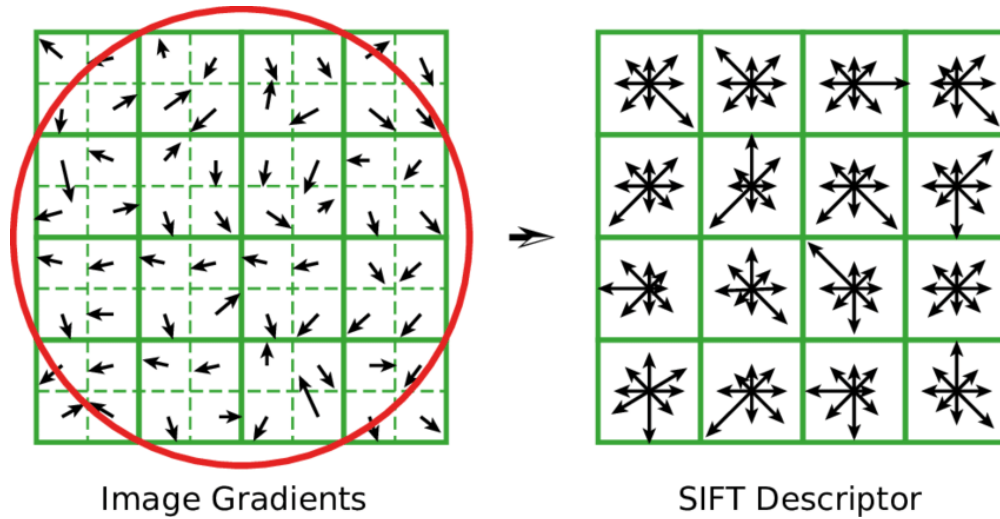


Figure A.2: Computing SIFT Descriptor.

## Appendix B

### Basic Relations For Calculating 3D Points and Pose

#### More on Epipolar Geometry and Epipolar Constraint

The figure B.1 shows the projection of point  $\mathbf{X}$  onto two separate camera planes whose optical centers are  $C$  and  $C'$  making images  $\mathbf{x}$  and  $\mathbf{x}'$  respectively in their planes. Now with image B.1 as reference,

- The **baseline** is the line joining the camera centers ( $CC'$ ).
- The **epipole** is the point of intersection of the baseline with the image plane ( $\mathbf{e}$  and  $\mathbf{e}'$ ). Equivalently, the **epipole** is the image in one view of the camera centre of the other view.
- An **epipolar plane** is a plane containing the baseline ( $\mathbf{X}CC'$ ).
- An **epipolar line** is the intersection of an epipolar plane with the image plane. ( $\mathbf{e}'\mathbf{x}'$  in second camera) All epipolar lines intersect at the epipole. An epipolar plane intersects the left and right image planes in epipolar lines, and defines the correspondence between the lines.

The correspondence of the two projected points is described by epipolar constraint. Epipolar constraint can be formulated algebraically using the essential matrix  $E$ , which relates corresponding image points in two views [11].

Consider two pinhole cameras as shown in figure B.1. Given a Euclidean point  $\mathbf{X}'$  in the coordinate system of camera  $C'$ , its position  $\mathbf{X}$  in the coordinate system of  $C$  is given by:

$$\mathbf{X} = \mathbf{R}\mathbf{X}' + \mathbf{T}$$

where  $R$  is a  $3 \times 3$  rotation matrix and  $T$  is a 3-vector.

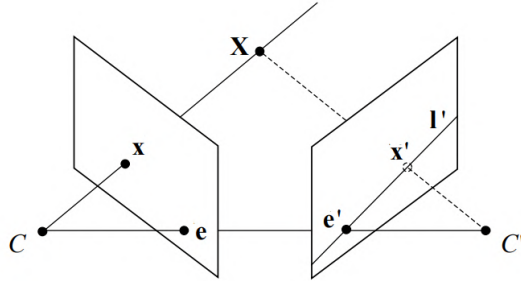


Figure B.1: Epipolar geometry for two cameras.

Pre-multiplying both sides by  $\mathbf{X}^T[\mathbf{T}]_{\times}$  gives:

$$\begin{aligned}\mathbf{X}^T[\mathbf{T}]_{\times}\mathbf{X} &= \mathbf{X}^T[\mathbf{T}]_{\times}\mathbf{R}\mathbf{X}, \\ \text{or, } \mathbf{X}^T\mathbf{E}\mathbf{X} &= 0\end{aligned}$$

where  $3 \times 3$  essential matrix  $\mathbf{E} \sim [\mathbf{T}]_{\times}\mathbf{R}$  and  $[\mathbf{T}]_{\times}$  is the cross product matrix. The above equation also holds for homogenous image points and is called epipolar constraint.

## More on Triangulation

Consider the images in figure B.1 as discussed in the epipolar geometry topic. Consider  $\mathbf{P}$  and  $\mathbf{P}'$  be the projection matrices for the first and second camera respectively. Here, a projection matrix is simply the product of the camera intrinsic matrix and the camera transformation matrix, i.e, the rotation and translation matrix, which results in a  $3 \times 4$  matrix.

Let  $\tilde{\mathbf{x}}$  and  $\tilde{\mathbf{x}}'$  represent homogenous coordinates corresponding to the image positions of the two cameras,  $\mathbf{x}$  and  $\mathbf{x}'$

Also let  $\tilde{\mathbf{X}}$  be the homogenous coordinate representing the 3D point  $\mathbf{X}$ .

So, the projection equations for the first, and then the second camera are,

$$\tilde{x} = \mathbf{P}\tilde{X}$$

$$\tilde{x}' = \mathbf{P}'\tilde{X}$$

Here, the only unknown is the four dimensional but homogenous coordinate  $\tilde{X}$ . Thus after expanding the equations, we get 3 unknowns and 4 equations. Expanding both of the equations in terms of components as ,

$$\tilde{\mathbf{x}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad \tilde{\mathbf{x}}' = \begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix} \quad \tilde{\mathbf{X}} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad \mathbf{P} = [m_{ij}] \quad \mathbf{P}' = [m'_{ij}]$$

we get a system of equation as follow:

$$\begin{bmatrix} um_{31} - m_{11} & um_{32} - m_{12} & um_{33} - m_{13} \\ vm_{31} - m_{11} & vm_{32} - m_{12} & vm_{33} - m_{13} \\ u'm'_{31} - m'_{11} & u'm'_{32} - m'_{12} & u'm'_{33} - m'_{13} \\ v'm'_{31} - m'_{11} & v'm'_{32} - m'_{12} & v'm'_{33} - m'_{13} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} m_{14} - m_{34} \\ m_{24} - m_{34} \\ m'_{14} - m'_{34} \\ m'_{24} - m'_{34} \end{bmatrix}$$

This system of equation can be solved easily using least square method using pseudo-inverse, which involves using SVD algorithm and we get the 3D coordinates of the point.

## Appendix C

### Solving Least Square

#### RANSAC

RANSAC is an iterative method of fitting a mathematical model to a set of observed data containing outliers. It is a learning technique based on random sampling of observed data. The input to this algorithm is a set of observed data values, a model to fit the observations, and some confidence parameters defining outliers. RANSAC follows roughly the following steps in achieving the goal :

1. Select a random sample of original data called ‘hypothetical inliers’.
2. Fit a model to the hypothetical inliers.
3. Test the whole data to the fitted model. The set of data points that fit the model well are called consensus set.
4. If sufficiently many data points have been a part of consensus set, then the estimated model is regarded as the final model, else choose this consensus set as the new set of hypothetical inliers and repeat the process.

#### Huber Loss Function

Huber loss is a loss function used in robust regression, that is less sensitive to outliers in data than the squared error loss. It is less sensitive to outliers and still maintains smoothness and differentiability. The loss function is defined by:

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2 & \text{for } |a| \leq \delta, \\ \delta \cdot (|a| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases}$$

This function is quadratic for small values of  $a$ , and linear for large values, with equal values and slopes of the different sections at the two points where  $|a| = \delta$ . The Huber loss can be visualized as the convolution of the absolute value function with the rectangular function, which is then scaled and translated. This convolution process effectively ”smoothens out” the sharp corner of the absolute value function at the origin.[12]

## TRF Least Square Minimization

TRF (Trust Region Reflective) least squares minimization is an effective algorithm for non-linear least squares problems. It adjusts model parameters iteratively to minimize the gap between predicted and observed values. Using a "trust region," it confines parameter updates to areas where the model approximates well. If the model performs satisfactorily, the region expands; otherwise, it contracts. By comparing predicted ( $\Delta f_{\text{pred}}$ ) and actual ( $\Delta f_{\text{actual}}$ ) reductions in the cost function, it optimizes the trust-region size ( $\lambda$ ) efficiently. Given  $\Delta x$ , we can evaluate

$$\Delta f_{\text{actual}} = f(x) - f(x + \Delta x)$$

.





## Appendix D

### Test Outputs

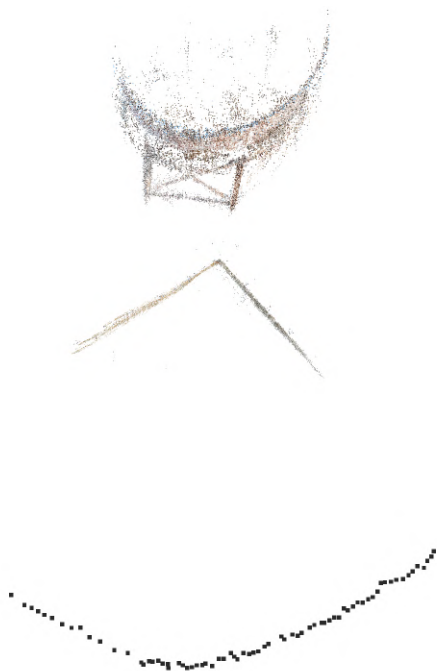
#### Proper Reconstructions



(a) Input image of water tower



(b) Front view for the reconstruction



(c) Top view with camera motion

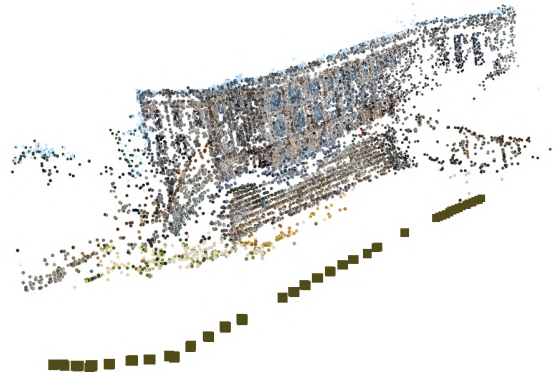


(d) Front View with camera motion

Figure D.1: Reconstructed water tower



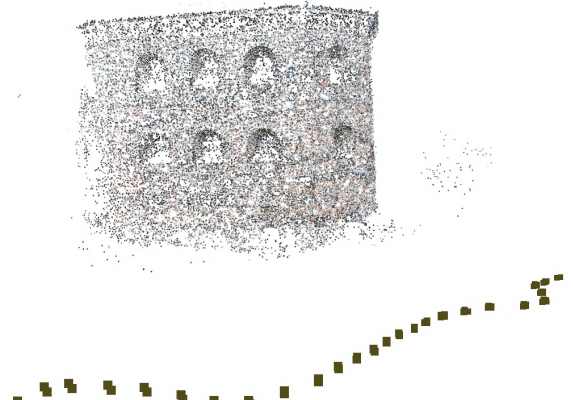
(a) Alcatraz Courtyard



(b) Reconstruction Alcatraz Courtyard



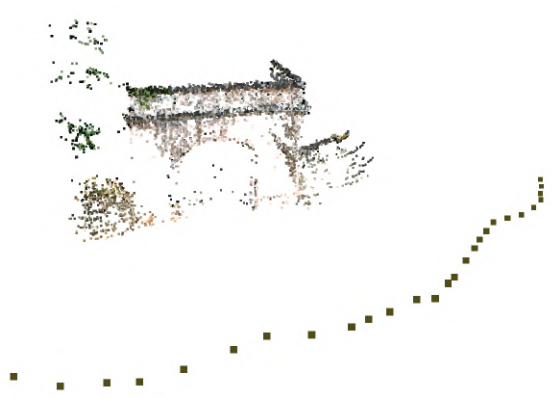
(a) Kronan



(b) Reconstruction Kronan



(a) Fort Chaining Gate



(b) Reconstruction Fort Chaining Gate



(a) Basilica di san petronio

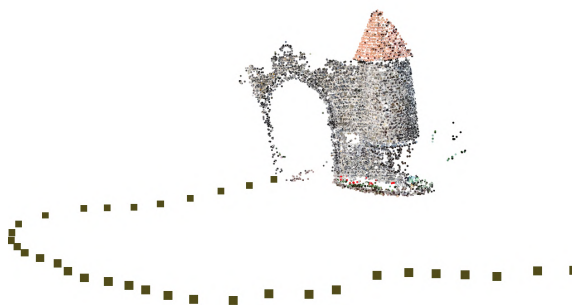


(b) Reconstruction Basilica di san petronio

z



(a) Park Gate



(b) Reconstruction Park Gate





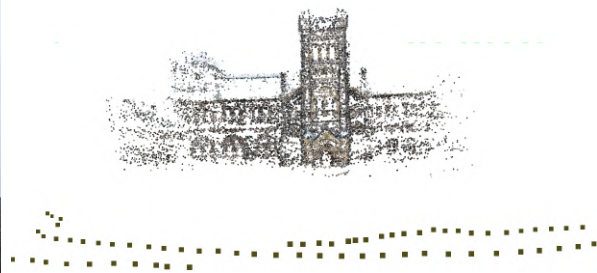
(a) Eglise



(b) Reconstruction Eglise



(a) UofT



(b) Reconstruction UofT

## Erroneous Reconstructions



(a) Room2



(b) Room2 Reconstruction with camera pose



(a) Room3



(b) Room3 Reconstruction with camera pose





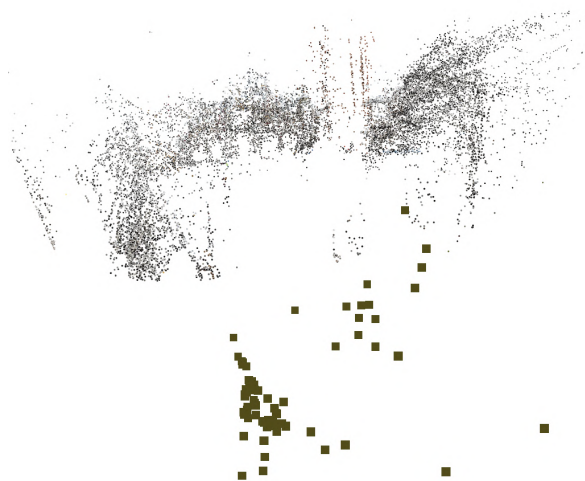
(a) Sri Thendayathapani



(b) Sri Thendayathapani Reconstruction with camera pose



(a) San Marco



(b) San Marco Reconstruction with camera pose

Table D.1: Dataset Output Table

| Dataset Name              | BA    | Images | Pixels | Time   | Features | 3D Points | Error Threshold | Reprojection Error |
|---------------------------|-------|--------|--------|--------|----------|-----------|-----------------|--------------------|
| Alcatraz_courtyard        | False | 78     | 9801   | 12.224 | 3417     | 350       | -               | 0.010              |
|                           | True  | 78     | 9801   | 11.883 | 3417     | 350       | 0.200           | 0.003              |
| Buddah_tooth_relic_temple | False | 33     | 9801   | 6.009  | 3743     | 137       | -               | 0.028              |
|                           | True  | 33     | 9801   | 5.809  | 3743     | 137       | 0.200           | 0.014              |
| Fort_Channing_gate        | False | 27     | 39204  | 4.123  | 4931     | 563       | -               | 0.009              |
|                           | True  | 27     | 39204  | 4.159  | 4931     | 563       | 0.200           | 0.002              |
| GustavIIAdolf             | False | 57     | 9801   | 5.378  | 4399     | 42        | -               | 0.018              |
|                           | True  | 57     | 9801   | 5.389  | 4399     | 42        | 0.200           | 0.007              |
| LUsphinx                  | False | 70     | 9801   | 11.084 | 8818     | 223       | -               | 0.009              |
|                           | True  | 70     | 9801   | 11.128 | 8818     | 223       | 0.200           | 0.004              |
| Sri_Thendayuthapani       | False | 80     | 9801   | 24.101 | 10070    | 308       | -               | 0.023              |
|                           | True  | 80     | 9801   | 23.911 | 10070    | 304       | 0.200           | 0.016              |
| Sri_Veeramakaliamman      | False | 50     | 9801   | 6.334  | 8739     | 171       | -               | 0.018              |
|                           | True  | 50     | 9801   | 6.080  | 8739     | 158       | 0.200           | 0.006              |
| St_Erik                   | False | 49     | 9801   | 9.085  | 5528     | 60        | -               | 0.019              |
|                           | True  | 49     | 9801   | 9.040  | 5528     | 60        | 0.200           | 0.002              |
| StatLib                   | False | 73     | 39204  | 64.264 | 7353     | 34        | -               | 0.042              |

Continued on next page



Table D.1: Dataset Output Table (Continued)

| Dataset Name             | BA    | Images | Pixels | Time   | Features | 3D Points | Error Threshold | Reprojection Error |
|--------------------------|-------|--------|--------|--------|----------|-----------|-----------------|--------------------|
|                          | True  | 73     | 39204  | 64.175 | 7353     | 34        | 0.200           | 0.042              |
| Thian_Hook_Keng_temple   | False | 30     | 39204  | 5.710  | 11686    | 543       | -               | 0.019              |
|                          | True  | 30     | 39204  | 5.691  | 11686    | 493       | 0.200           | 0.003              |
| UofT                     | False | 77     | 9801   | 9.910  | 5033     | 202       | -               | 0.032              |
|                          | True  | 77     | 9801   | 10.037 | 5033     | 202       | 0.200           | 0.029              |
| Yueh_Hai_Ching_Temple    | False | 43     | 9801   | 5.802  | 5964     | 195       | -               | 0.024              |
|                          | True  | 43     | 9801   | 5.901  | 5964     | 187       | 0.200           | 0.013              |
| ahlstromer               | False | 40     | 39204  | 5.289  | 8782     | 205       | -               | 0.021              |
|                          | True  | 40     | 39204  | 5.231  | 8782     | 205       | 0.200           | 0.012              |
| alcatraz_reduced         | False | 55     | 9801   | 7.542  | 2665     | 408       | -               | 0.008              |
|                          | True  | 55     | 9801   | 7.652  | 2665     | 408       | 0.200           | 0.005              |
| barcelona_cath           | False | 54     | 9801   | 8.144  | 3096     | 26        | -               | 0.061              |
|                          | True  | 54     | 9801   | 7.906  | 3096     | 26        | 0.200           | 0.024              |
| basilica_di_san_petronio | False | 46     | 9801   | 12.157 | 3151     | 332       | -               | 0.010              |
|                          | True  | 46     | 9801   | 11.940 | 3151     | 325       | 0.200           | 0.007              |
| buddah1                  | False | 72     | 9801   | 7.695  | 5957     | 53        | -               | 0.019              |
|                          | True  | 72     | 9801   | 7.540  | 5957     | 53        | 0.200           | 0.012              |

Continued on next page

Table D.1: Dataset Output Table (Continued)

| Dataset Name    | BA    | Images | Pixels | Time   | Features | 3D Points | Error Threshold | Reprojection Error |
|-----------------|-------|--------|--------|--------|----------|-----------|-----------------|--------------------|
| c_door          | False | 56     | 12288  | 25.210 | 3869     | 38        | -               | 0.036              |
|                 | True  | 56     | 12288  | 24.919 | 3869     | 38        | 0.200           | 0.012              |
| carlsten        | False | 77     | 39204  | 11.289 | 9265     | 513       | -               | 0.014              |
|                 | True  | 77     | 39204  | 11.566 | 9265     | 504       | 0.200           | 0.014              |
| council_chamber | False | 80     | 9801   | 15.257 | 8477     | 521       | -               | 0.036              |
|                 | True  | 80     | 9801   | 15.457 | 8477     | 521       | 0.200           | 0.004              |
| de_guerre       | False | 35     | 9801   | 4.149  | 3029     | 263       | -               | 0.027              |
|                 | True  | 35     | 9801   | 4.176  | 3029     | 263       | 0.200           | 0.017              |
| dome            | False | 61     | 39204  | 7.660  | 10284    | 223       | -               | 0.017              |
|                 | True  | 61     | 39204  | 7.594  | 10284    | 223       | 0.200           | 0.007              |
| door            | False | 12     | 9801   | 1.602  | 685      | 175       | -               | 0.014              |
|                 | True  | 12     | 9801   | 1.641  | 685      | 175       | 0.200           | 0.007              |
| eglise          | False | 80     | 9801   | 10.328 | 4905     | 107       | -               | 0.017              |
|                 | True  | 80     | 9801   | 10.296 | 4905     | 107       | 0.200           | 0.010              |
| filbyter        | False | 40     | 156816 | 10.359 | 33487    | 159       | -               | 0.235              |
|                 | True  | 40     | 156816 | 10.355 | 33487    | 159       | 0.200           | 0.223              |
| gbd_reduced     | False | 67     | 9801   | 12.252 | 2565     | 220       | -               | 0.011              |

Continued on next page

Table D.1: Dataset Output Table (Continued)

| Dataset Name           | BA    | Images | Pixels | Time   | Features | 3D Points | Error Threshold | Reprojection Error |
|------------------------|-------|--------|--------|--------|----------|-----------|-----------------|--------------------|
|                        | True  | 67     | 9801   | 12.001 | 2565     | 212       | 0.200           | 0.004              |
| golden_statue          | False | 18     | 9801   | 2.155  | 2333     | 202       | -               | 0.004              |
|                        | True  | 18     | 9801   | 2.076  | 2333     | 202       | 0.200           | 0.002              |
| house                  | False | 12     | 39204  | 3.710  | 3128     | 268       | -               | 0.004              |
|                        | True  | 12     | 39204  | 3.703  | 3128     | 268       | 0.200           | 0.002              |
| kings_college_interior | False | 63     | 9801   | 9.203  | 5334     | 594       | -               | 0.004              |
|                        | True  | 63     | 9801   | 9.018  | 5334     | 579       | 0.200           | 0.002              |
| lund_cath_small        | False | 17     | 9801   | 14.414 | 2717     | 53        | -               | 0.025              |
|                        | True  | 17     | 9801   | 14.428 | 2717     | 53        | 0.200           | 0.025              |
| nijo                   | False | 19     | 39204  | 10.057 | 5992     | 663       | -               | 0.012              |
|                        | True  | 19     | 39204  | 9.962  | 5992     | 655       | 0.200           | 0.006              |
| nikolaiI               | False | 80     | 9801   | 9.460  | 8389     | 250       | -               | 0.008              |
|                        | True  | 80     | 9801   | 9.249  | 8389     | 250       | 0.200           | 0.003              |
| park_gate              | False | 34     | 9801   | 9.800  | 4398     | 39        | -               | 0.003              |
|                        | True  | 34     | 9801   | 9.811  | 4398     | 39        | 0.200           | 0.001              |
| pumpkin                | False | 64     | 9801   | 8.977  | 5919     | 37        | -               | 0.048              |
|                        | True  | 64     | 9801   | 8.816  | 5919     | 37        | 0.200           | 0.015              |

Continued on next page

Table D.1: Dataset Output Table (Continued)

| Dataset Name     | BA    | Images | Pixels | Time   | Features | 3D Points | Error Threshold | Reprojection Error |
|------------------|-------|--------|--------|--------|----------|-----------|-----------------|--------------------|
| reduced_pulchowk | False | 55     | 3072   | 31.162 | 2485     | 38        | -               | 0.025              |
|                  | True  | 55     | 3072   | 31.082 | 2485     | 38        | 0.200           | 0.012              |
| room1            | False | 39     | 9801   | 7.629  | 3111     | 81        | -               | 0.026              |
|                  | True  | 39     | 9801   | 7.591  | 3111     | 81        | 0.200           | 0.014              |
| room2            | False | 50     | 39204  | 14.743 | 9915     | 1750      | -               | 0.021              |
|                  | True  | 50     | 39204  | 14.410 | 9915     | 1694      | 0.200           | 0.010              |
| room3            | False | 54     | 9801   | 15.011 | 6642     | 425       | -               | 0.009              |
|                  | True  | 54     | 9801   | 15.019 | 6642     | 425       | 0.200           | 0.004              |
| room4            | False | 52     | 9801   | 6.210  | 5623     | 417       | -               | 0.010              |
|                  | True  | 52     | 9801   | 6.191  | 5623     | 417       | 0.200           | 0.002              |
| room5            | False | 23     | 9801   | 4.074  | 2792     | 312       | -               | 0.006              |
|                  | True  | 23     | 9801   | 4.046  | 2792     | 312       | 0.200           | 0.004              |
| round_church     | False | 75     | 9801   | 23.389 | 7366     | 122       | -               | 0.014              |
|                  | True  | 75     | 9801   | 23.333 | 7366     | 122       | 0.200           | 0.003              |
| san_marco1       | False | 67     | 39204  | 10.508 | 16810    | 1077      | -               | 0.028              |
|                  | True  | 67     | 39204  | 10.348 | 16810    | 964       | 0.200           | 0.013              |
| smolny           | False | 71     | 9801   | 14.385 | 6127     | 567       | -               | 0.008              |

Continued on next page

Table D.1: Dataset Output Table (Continued)

| Dataset Name  | BA    | Images | Pixels | Time    | Features | 3D Points | Error Threshold | Reprojection Error |
|---------------|-------|--------|--------|---------|----------|-----------|-----------------|--------------------|
|               | True  | 71     | 9801   | 14.212  | 6127     | 567       | 0.200           | 0.005              |
| staircase1    | False | 10     | 9801   | 2.817   | 747      | 168       | -               | 0.004              |
|               | True  | 10     | 9801   | 2.777   | 747      | 168       | 0.200           | 0.003              |
| urbanII       | False | 80     | 156816 | 47.665  | 70343    | 7077      | -               | 1.756              |
|               | True  | 80     | 627264 | 465.614 | 232431   | 17296     | 0.200           | 0.103              |
| uwo           | False | 57     | 39204  | 9.879   | 10200    | 470       | -               | 0.010              |
|               | True  | 57     | 39204  | 9.996   | 10200    | 470       | 0.200           | 0.004              |
| vasa_statue   | False | 18     | 9801   | 1.905   | 2449     | 50        | -               | 0.018              |
|               | True  | 18     | 9801   | 1.879   | 2449     | 50        | 0.200           | 0.018              |
| vercingetorix | False | 69     | 9801   | 10.635  | 4140     | 39        | -               | 0.026              |
|               | True  | 69     | 9801   | 10.621  | 4140     | 39        | 0.200           | 0.018              |
| ystad_kloster | False | 42     | 9801   | 6.266   | 3458     | 94        | -               | 0.029              |
|               | True  | 42     | 9801   | 6.039   | 3458     | 94        | 0.200           | 0.022              |

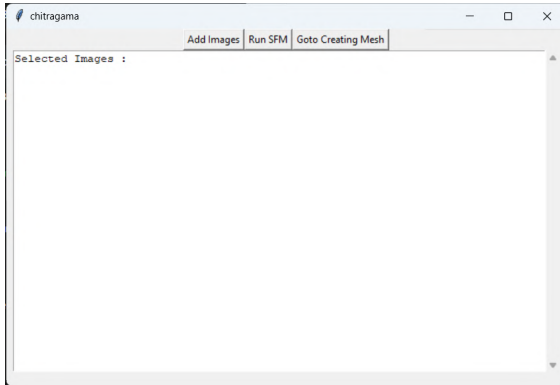
## Appendix E

### Using The Software

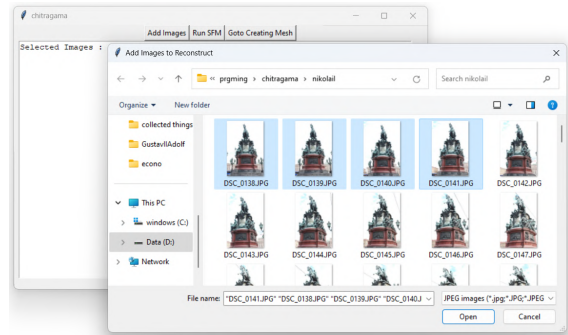
The user interface of our application is very simple, as we require very less interaction from the user for executing the SfM pipeline. There are three buttons on the main screen of our application, one for adding the image, another for running SfM on the images, and final for skipping the SfM pipeline and running the mesh generation. Following sections depict some screen shots of working with our chitragama application.

#### Using SfM Pipeline

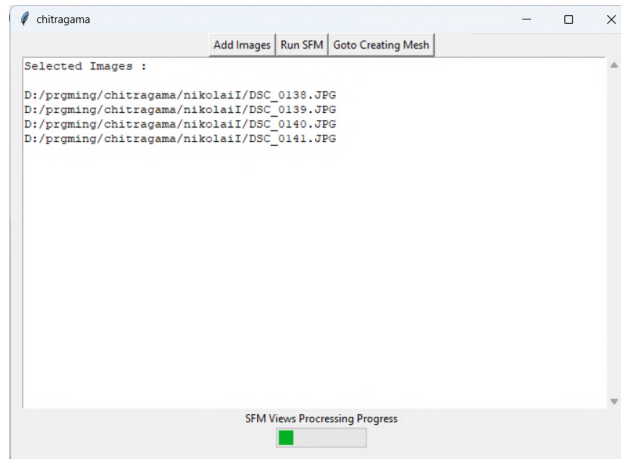
The images shown here depict us working with the application to produce a sparse point cloud and the associated camera path as a .ply file.



(a) Initial Window



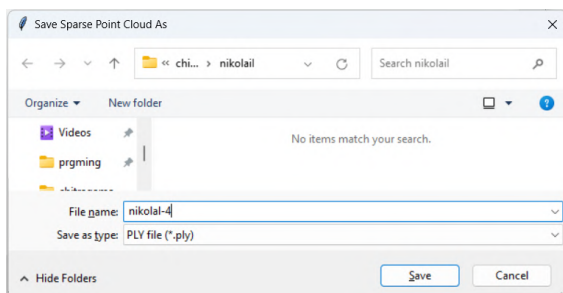
(b) Adding Images to Process



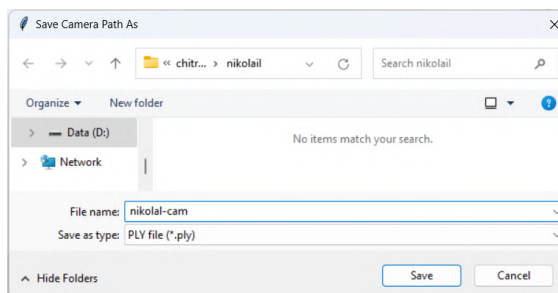
(c) Processing Images

Figure E.1: Initiating SfM

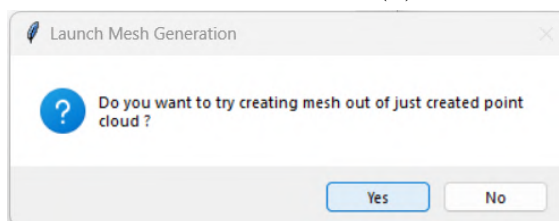
Here we also show some dialog boxes to save the point cloud as file and we launch the output of SfM and feed it to the mesh reconstruction component, initially showing the point cloud generated.



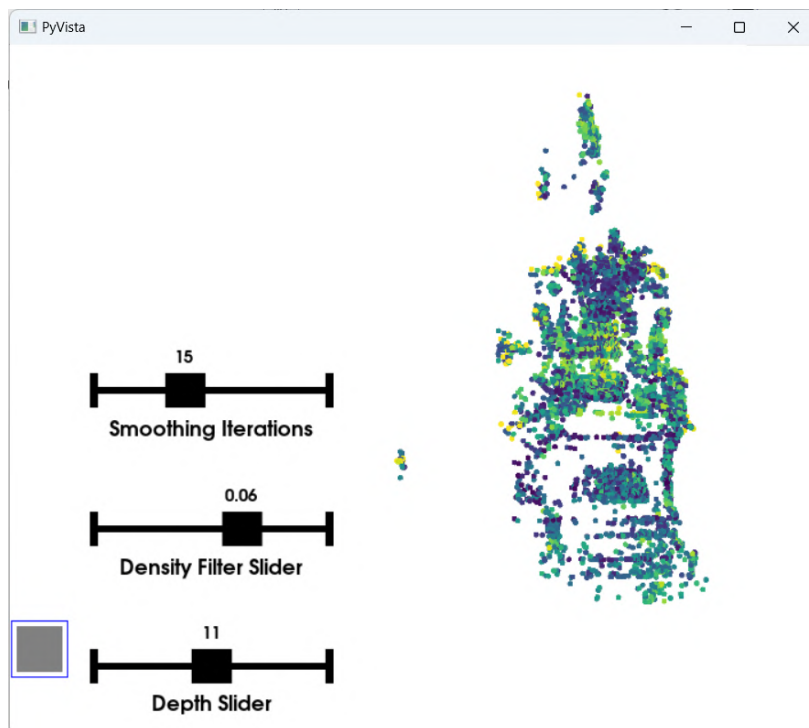
(a) Save as Dialog for Point Cloud



(b) Save as Dialog for Camera Path



(c) Dialog for Launch Mesh Generation Directly

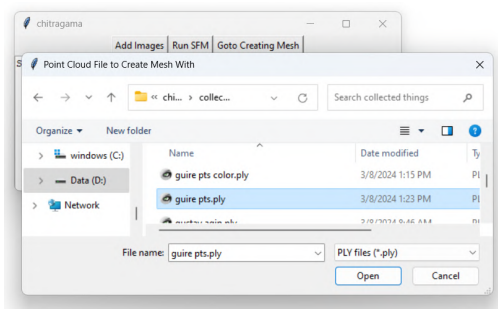


(d) Mesh Generation from SfM Output

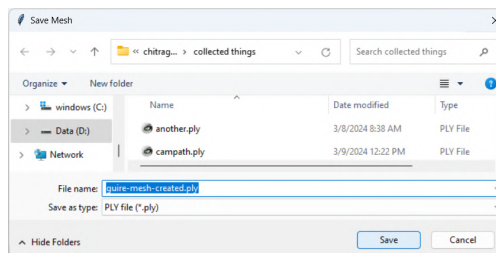
Figure E.2: After Completion of SfM

## Using Surface Reconstruction

Here are some shots of our application performing some mesh reconstruction of point cloud built using a large number of images. For that purpose, we choose a .ply file previously produced using the SfM pipeline. We also see the result through an external program, meshlab.

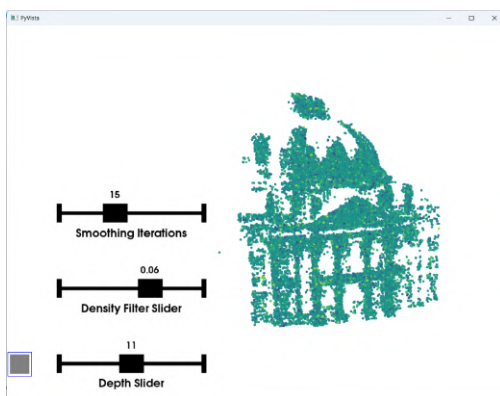


(a) Opening Point Cloud File

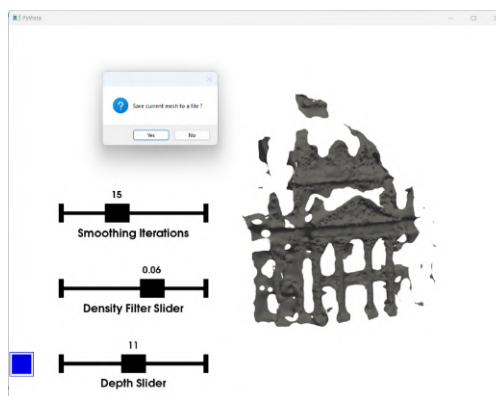


(b) Save as Dialog for Mesh

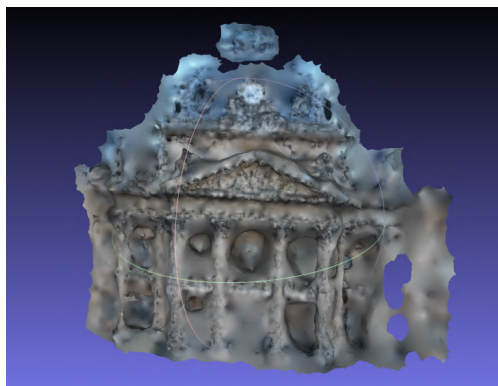
Figure E.3: Mesh Generation File Dialog Boxes



(a) Before Mesh Generation



(b) After Mesh Generation



(c) Output in Mesh Lab

Figure E.4: Mesh Generation Input and Output